Magic Pot*: Cryptanalysis of full AIM2 in the standard and related-/reused-key settings using new elimination framework**

Alex Biryukov[1,2], Pablo García Fernández[2], and Aleksei Udovenko[2]

DCS, University of Luxembourg, Esch-sur-Alzette, Luxembourg
SnT, University of Luxembourg, Esch-sur-Alzette, Luxembourg
first-name.last-name@uni.lu

Abstract. In this work, we cryptanalyse the post-quantum signature scheme AIMer v2.1, which is one of the winners of the Korean Post-Quantum Cryptography competition (KpqC), and whose earlier version was a candidate in the US NIST's additional post-quantum digital signatures call. We show that AIM2, the underlying symmetric-key primitive, is not secure up to the claimed level by developing and applying a new algebraic attack framework based on extended linearization over a univariate polynomial ring and a novel algorithm for finding a null vector of a polynomial matrix. In particular misuse scenarios, such as reused-key or related-key settings, our attacks become practically feasible, allowing experimental verification and benchmarking. We also evaluate the approach on the RAIN block cipher used in the Rainier post-quantum signature scheme and obtain improved attacks, although not threatening its claimed security.

Keywords: Post-quantum · Cryptanalysis · AIMer · AIM2 · RAIN · Related-key · Reused-key · Algebraic attacks · Linearization · XL · Elimination · Polynomial matrices

1 Introduction

The potential development of quantum computers in the near future has spurred efforts to develop and standardize quantum-resistant cryptographic primitives. In particular, the US NIST organized a Post-Quantum Cryptography (PQC) standardization process in 2017, which led to the selection of several lattice-based [SAB+22,LDK+22], code-based [AAB+22] and hash-based [HBD+22] key encapsulation schemes and digital signatures in 2024 and 2025. An additional call for digital signature schemes that focus on alternative paradigms was announced, and the selection process is currently in the second round. Similarly, in 2021, South Korean institutional bodies created a Korean Post-Quantum Cryptography (KpqC) research group that held a competition between 2022 and 2025, resulting in a selection of lattice-based [CCD+24,KP22,CCHY24] and symmetric-key-based [KHS+24] schemes.

AIMer [KHS+24] is a post-quantum signature scheme based on the MPC-in-the-Head paradigm. It relies on the security of the underlying symmetric-key primitive, the block cipher AIM/AIM2. The first version of AIMer, initially published in CCS 2023 [KHS+23], was a first-round candidate of the NIST additional signature call,[1] but did not advance further due to cryptanalysis reports [LMØM23,ZWY+23]. The designers mitigated the attacks [KHSL23] and proposed a new version of the scheme, based on the modified underlying block cipher AIM2. This scheme became one of the winners of the KpqC competition.[2]

Rainier is a similar scheme published in CCS 2022 [DKR+22], inspiring AIMer. It has not yet appeared in standardization competitions. It follows the same design paradigm (MPC-in-the-head) and principles. Due to the similarity of the designs, they are typically analyzed together.

The importance of the MPC-in-the-Head paradigm is that it provably reduces security of the signature to the security of the underlying symmetric-key primitive, where security is believed to be better understood. Nevertheless, the paradigm forces many nonstandard design choices, compared to traditional symmetric-

* Starting with a "pot" of just three equations, say "Cook, little pot, cook" and a lot of new equations are produced. The key is to know the magic phrase to stop cooking (i.e. when the system reaches the full rank).
** Full version of the paper accepted at EUROCRYPT 2026.
[1] https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures
[2] https://www.kpqc.or.kr/competition_02.html

key cryptography. In this work, we will show that AIMer/AIM2 has significant weaknesses which can be exploited by a new algebraic attack framework.

Reused-key and related-key settings AIM2 is parametrized by an initialization vector (IV) intended to increase multi-user security of the scheme. One may consider a *misuse* scenario of reusing the same secret key across different IVs. While such an attack is known to be out of scope,[3] it provides an interesting security benchmark and allows practical verification of our theory. In addition, we show that a *related-key* setting with a specially crafted affine relation over $\mathbb{F}_2$ leads to the same weakness, independently of whether IVs are changed or not.

These scenarios could take place in the case of a user (or a vendor) misusing the full signature scheme by generating secret keys and/or IVs in a custom (or even subverted) manner. A notorious real-world example is the ROCA vulnerability [NSS+17] that affected millions of smart cards: a company generated secret keys using a custom method for efficiency reasons, allowing their practical recovery purely from the corresponding public keys. It may also be relevant in the side-channel setting, where a change of IV/key could be achieved by a fault attack.

Our contributions

(New algebraic elimination framework) We develop a new algebraic attack framework similar to XSL [CP02] (eXtended Sparse Linearization) and related to the Free-Lunch approach [BBL+24]. One of the key ideas is performing partial linearization excluding one chosen variable, which allows us to formulate the problem in terms of polynomial matrices and apply powerful algorithms. The framework has two important advantages.

First, the requirements of our framework are simpler than in similar works. We do not require any monomial order, Gröbner basis, shape position, stability, quotient ring computations, multiplication matrix construction, etc. The only requirement is that a small part of the equation system expanded by multiplications has full rank, which can often be verified in practice (in contrast to XL variants that need verification of the full system to ensure validity of attacks).

Second, the linear algebraic complexity of elimination is reduced to *quasilinear* in the degree of the chosen variable, while staying ω (the matrix multiplication exponent) for the degrees of the remaining variables, or quadratic in the case of sparse systems. The latter can be achieved by state-of-the-art polynomial matrix algorithms or by our *new tailored algorithm* for computing a kernel vector of a sparse polynomial matrix, which is simpler to implement and analyze while achieving state-of-the-art complexity on very sparse systems arising in our work.

(Theoretic and practical attacks on AIMer v2.1 / AIM2) We apply our framework to full AIM2 and obtain provable and validated theoretic attacks in the *standard setting*, as well as practical attacks in the *reused-key/related-key* models, fully verified experimentally for as few as 11 IVs/related-keys. More specifically, by reused keys we mean a number of public-keys generated from the same secret key but different IVs; by related keys we mean a number of public-keys whose secret keys are related by a specific bitwise-affine relation in the finite field and IVs are arbitrary (fixed or random). Both cases have an equivalent impact depending only on the number of available public keys.

Our main attack on AIM2 with L IVs (including $L = 1$ for the standard model) has complexity being a small multiple of the cost of finding univariate roots of a degree-ξ polynomial, implying roughly an $\ell/(\ell + L)$ reduction in bit security:

$$\xi = \mathcal{O}\left(2^{\frac{\ell n}{\ell+L}}(\ell + L)\right) = \mathcal{O}(2^{n \cdot \frac{\ell}{\ell+L}}).$$

(Improved attacks on RAIN) We also apply our approach to RAIN. For the reduced 2-round version, our attack is comparable to the very recent best attack [YZY26]. For the full 3 rounds, our attack is the best among algebraic approaches in the literature, although it does not beat the exhaustive search.

[3] In AIM2, the roles of the plaintext and the IV are similar to the roles, respectively, of the secret key and the plaintext in more traditional designs such as Picnic [ZCD+20], potentially causing more confusion.

Target	n	ℓ/r	Model	Time	Memory	Ref.
AIM2						
AIM2-I	128	2	Reuse, 109 IVs	$2^{23.36}$ (23min)†	$2^{22.17}$	Subsection 2.5
AIM2-I	128	2	Reuse, 11 IVs	$2^{36.01}$ (33h)*	$2^{36.53}$	Subsection 4.2
AIM2-I	128	2	Standard	$2^{125.8}$	negl.	[BCG^{+}24]
AIM2-I	128	2	Standard	$2^{103.66}$	$2^{100.33}$	Subsection 4.2
AIM2-III	192	2	Reuse, 163 IVs	$2^{24.52}$ (80min)†	$2^{23.44}$	Subsection 2.5
AIM2-III	192	2	Reuse, 20 IVs	$2^{34.72}$ (30h)*	$2^{36.22}$	Subsection 4.2
AIM2-III	192	2	Standard	$2^{189.9}$	negl.	[BCG^{+}24]
AIM2-III	192	2	Standard	$2^{147.46}$	$2^{144.17}$	Subsection 4.2
AIM2-V	256	3	Reuse, 327 IVs	$2^{27.1}$ (11h)†	$2^{25.52}$	Subsection 2.5
AIM2-V	256	3	Reuse, 52 IVs	$2^{33.69}$ (37h)*	$2^{35.36}$	Subsection 4.2
AIM2-V	256	3	Standard	$2^{254.4}$	negl.	[BCG^{+}24]
AIM2-V	256	3	Standard	$2^{213.19}$	$2^{209.91}$	Subsection 4.2
RAIN						
RAIN-128-2	128	2	Standard	$2^{120.3}$	$2^{14.32}$	[ZWY^{+}23]
RAIN-128-2	128	2	Standard	2^{90}	2^{42}	[LMØM23]
RAIN-128-2	128	2	Standard	$2^{73.7}$	?	[YZY26]
RAIN-128-2	128	2	Standard	$2^{80.04}$	$2^{72.02}$	Section 5
RAIN-192-2	192	2	Standard	2^{107}	?	[YZY26]
RAIN-192-2	192	2	Standard	$2^{113.16}$	$2^{105.18}$	Section 5
RAIN-256-2	256	2	Standard	$2^{138.9}$	?	[YZY26]
RAIN-256-2	256	2	Standard	$2^{145.97}$	$2^{138.01}$	Section 5
RAIN-128-3	128	3	Standard	2^{132} ($\omega = 2$)	?	[LMM24]
RAIN-128-3	128	3	Standard	2^{166} ($\omega = 2.8$)	?	[LMM24]
RAIN-128-3	128	3	Standard	$2^{160.6}$	?	[YZY26]
RAIN-128-3	128	3	Standard	$2^{139.07}$	$2^{134.78}$	Section 5
RAIN-192-3	192	3	Standard	$2^{188.42}$ ($\omega = 2$)	?	[LMM24]
RAIN-192-3	192	3	Standard	$2^{266.42}$ ($\omega = 2.8$)	?	[LMM24]
RAIN-192-3	192	3	Standard	$2^{236.02}$	?	[YZY26]
RAIN-192-3	192	3	Standard	$2^{200.66}$	$2^{196.37}$	Section 5

Table 1: Summary of our and previous attacks against AIM2 and RAIN. All primitives (except 2-round-reduced RAIN) have claimed security levels of n bits. Time complexity is in number of encryptions approximated as n field operations as in previous works, memory usage is in bytes ($n/8$ per field element). †Timing using our unoptimized SageMath implementation of the simple linearization key recovery attack from Subsection 2.5. *Timing using our optimized SageMath/Rust implementation of the polynomial matrix elimination attack from Subsection 4.2.

Our attacks are summarized and compared to previous best attacks in Table 1.

Experimental setup All experiments in this work were performed on a laptop with an Intel i7 3GHz CPU and 32 GB RAM, and, for the purpose of batch computations and a few cases requiring large memory amounts, on a server with an AMD EPYC 3.2GHz CPU and 1 TB RAM. We developed a library in Rust with efficient algorithms for binary finite fields, polynomials, matrices, and linear algebra. The library is called from SageMath [SD25] through Python bindings. Source code will be made publicly available [BGFU26].[4]

Previous work Our main inspiration comes from the well-known XL/XSL methods [CKPS00,CP02], over which we improve both validation and solving complexity. Univariate polynomial solving was applied to algebraic primitives in [BBLP22]. More recent methods are the algebraic FreeLunch [BBL+24] and a related theoretic approach from [BNSED22]. Both works develop algorithms for change of monomial order in the Gröbner basis and compute determinant of a certain multiplication matrix in the quotient ring. Our approach turns out to be closely related (similar complexity, connection to the determinant), but works outside of the Gröbner basis setting, making it even more applicable.

Several works studied the first version of AIM [LMØM23,ZWY+23,YZY26] based on equivalent representations, linearization, polynomial decomposition, and resultants. They also break 2 rounds of RAIN with varying complexities. Notably, [LMM24] speculates about a potential XL attack on 3-round RAIN, but its validity is not established. To our knowledge, the only work that analyzes the current version v2.1 (AIM2), the KpqC winner, is [BCG+24]. Their approach is to construct an explicit quadratic (over $\mathbb{F}_2$) system, guess an intermediate n-bit value, and solve the remaining structured linear system faster than one AIM2 evaluation. Under matrix multiplication exponent $\omega = 2.37$, this results in a small improvement over the expected security levels (around 3 bits, see Table 3).

Paper structure The structure of this paper is as follows. Section 2 is dedicated to the presentation of the AIM scheme, as well as the algebraic modeling and other ideas on which we will work. In particular, Subsection 2.5 presents a first attack on AIM. Section 3 presents our new algebraic elimination framework, which is then applied to AIM in Section 4 and to RAIN in Section 5. Finally, Section 6 discusses the possible reparation of AIM2, presents future work, and concludes this paper.

Notation and preliminaries We write $\mathbb{F}$ for any finite field, $\overline{\mathbb{F}}$ for its algebraic closure, $\mathbb{F}[x]$ for its ring of polynomials, $\mathbb{F}[x]_{\leq d}$ for its set of polynomials of degree at most d, and $\mathbb{F}_{2^n}$ for the finite field of size 2^n. We denote by HW the Hamming weight (either of a vector, or of the binary representation of an integer). We use the $\sim$-notation for complexity, which specifies the leading asymptotic constant. The matrix multiplication exponent is denoted by ω ($\omega = 2.8$ using Strassen's method).

The cost of multiplying two degree-d polynomials is denoted $\mathbf{M}(d)$. For $\mathbb{F}_2^n$, we use $\mathbf{M}(d) \sim 0.5d(\log_2 d)^2 + \mathcal{O}(d \log d)$ from Gao-Mateer's additive FFT [GM10], and we use an implementation based on Frobenius Additive FFT (FAFFT) [LCK+18]. HalfGCD of two non-constant degree-d polynomials $f(x), g(x)$ returns the middle Euclidean remainder $r(x), \deg r(x) < d/2$ and its polynomial Bézout coefficients $u(x), v(x)$ of degree at most $\lceil d/2 \rceil$: $u \cdot f + v \cdot g = r$. In particular, $r \neq 0$ if f and g are coprime. The complexity cost of HalfGCD is $\sim 5.5\mathbf{M}(d)\log_2(d)$, according to [vdH25] (the general FFT case). Assuming $f(x) \in \mathbb{F}[x]_{\leq d}$ has a small number of roots in $\mathbb{F}$, the cost of finding them is given by $\sim \mathbf{M}(d)(6\log_2|\mathbb{F}| + 11\log_2 d)$. If $\mathbb{F} = \mathbb{F}_{2^n}$, the cost reduces to $\sim \mathbf{M}(d)(2n + 11\log_2 d)$. For a more detailed survey and analysis of the above costs, we refer to Section A.

2 Description and algebraic model of AIM2

We will consider equations that describe the computation of AIM2 in the native field $\mathbb{F}_{2^n}$. This contrasts with the analysis by the designers [KHS+23,KHS+24] which was done by modeling AIM/AIM2 over $\mathbb{F}_2$, motivated by the fact that the S-boxes have a quadratic algebraic degree. Furthermore, the random $\mathbb{F}_2$-linear layer Lin has degree 2^{n-1} when expressed over $\mathbb{F}_{2^n}$, which makes any standard algebraic analysis infeasible. Although [YZY26] recently proposed a method to represent the linear layer using a composition of two lower-degree linearized polynomials, the technique was applied only to AIM and not to AIM2.

[4] See also https://github.com/cryptolu/AIM2_cryptanalysis

2.1 Description of AIM2

The AIMer v2.1 signature scheme is based on a primitive called AIM2 which has three instances: I (128 bits of security), III (192 bits), and V (256 bits). The public key is made up of a (ciphertext, initialization vector) pair, (c, iv), where iv is chosen by the user and determines the linear layer, and c is the result of applying the cipher to the private key p with the linear layer given by iv.

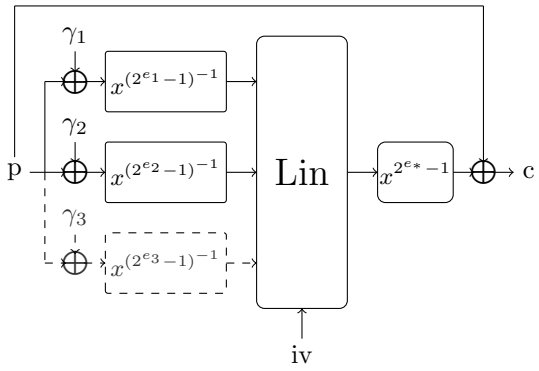


Fig. 1: The structure of AIM.

Variant	n	ℓ	e_1	e_2	e_3	e_*
I	128	2	49	91	-	3
III	192	2	17	47	-	5
V	256	3	11	141	7	3

Table 2: Parameters of AIM2.

We will denote by n the respective security level, which also determines the finite field where the scheme is defined on ($\mathbb{F}_{2^n}$). The three instances have the structure depicted in Figure 1 and the exponents summarized in Table 2. Here, $p \in \mathbb{F}_{2^n}$ is the secret key, $\gamma_i \in \mathbb{F}_{2^n}$ are fixed constants independent of iv, $x^{(2^{e_i}-1)^{-1}}$ are monomial S-boxes (the inversion is done modulo $2^n - 1$), $\ell \in \{2, 3\}$ is the number of branches in the first S-box layer, $\mathrm{Lin} : (\mathbb{F}_{2^n})^\ell \to \mathbb{F}_{2^n}$ is the internal affine layer generated pseudorandomly from the iv, c is the "ciphertext" which is the main part of the signature scheme's public key. The generation of the affine layer requires Lin to consist of ℓ bijective linear components L_i and a constant r: $\mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell) = \mathrm{L}_1(\mathrm{t}_1) + \ldots + \mathrm{L}_\ell(\mathrm{t}_\ell) + \mathrm{r}$.

2.2 Basic model and reused keys

We will modelize and cryptanalyze AIM over the full finite field $\mathbb{F}_{2^n}$. Our basic algebraic model will closely follow the computational process described in Subsection 2.1, with a few minor modifications. We will use variable $p \in \mathbb{F}_{2^n}$ for the secret key (plaintext), and variables for intermediate values $\mathbf{t} = (t_1, \ldots, t_\ell) \in (\mathbb{F}_{2^n})^\ell$ corresponding to the outputs of the first layer of monomial S-boxes. By definition, we have

$$t_i = (p + \gamma_i)^{(2^{e_i}-1)^{-1}} \quad \Leftrightarrow \quad t_i^{2^{e_i}-1} = p + \gamma_i.$$

Furthermore, by $\mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell)$ we denote the expression of the linear layer as a linearized polynomial over $\mathbb{F}_{2^n}$, that is,

$$\mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell) = \lambda_0 + \sum_{\mathrm{i}=0}^{\mathrm{n}-1} (\lambda_{1,\mathrm{i}} \mathrm{t}_1^{2^\mathrm{i}} + \lambda_{2,\mathrm{i}} \mathrm{t}_2^{2^\mathrm{i}} + \ldots + \lambda_{\ell,\mathrm{i}} \mathrm{t}_\ell^{2^\mathrm{i}}), \quad \lambda_0, \lambda_{\mathrm{j,i}} \in \mathbb{F}_{2^\mathrm{n}}.$$

This defines the final piece of the computational description as

$$\mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell)^{2^{\mathrm{e}_*}-1} = \mathrm{p} + \mathrm{c}.$$

Our next step is to bring the equations into quadratic form (in terms of the algebraic degree) by multiplying them by t_i and $\mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell)$. We obtain the following formulation.

Proposition 1. *Let F be an instance of AIM2 and let $c \in \mathbb{F}_{2^n}$ be fixed. Then, any solution $p', t_1, \ldots, t_\ell \in \mathbb{F}_{2^n}$ with $t_1 \neq 0, \ldots, t_\ell \neq 0, \mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell) \neq 0$ in the variety of the ideal $I = \langle f_1, f_2, \ldots, f_{\ell+1} \rangle \subseteq \mathbb{F}_{2^n}[\mathbf{t}, p]$,*

$$f_1 = (p + \gamma_1) \cdot t_1 - t_1^{2^{e_1}}, \tag{1}$$

$$f_2 = (p + \gamma_2) \cdot t_2 - t_2^{2^{e_2}}, \tag{2}$$

$$f_3 = (p + \gamma_3) \cdot t_3 - t_3^{2^{e_3}}, \quad \textit{(when } \ell = 3\textit{)} \tag{3}$$

$$f_{\ell+1} = (p + c) \cdot \mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell) - \mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell)^{2^{\mathrm{e}_*}} \tag{4}$$

corresponds to a valid preimage p' satisfying $c = F(p')$. Conversely, for any such valid preimage p' there exists a valid assignment for $t_1, \ldots, t_\ell \in \mathbb{F}_{2^n}$ either belonging to the variety or having one of $t_1, \ldots, t_\ell, \mathrm{Lin}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell)$ equal to zero.

Reused-key model Following the discussion from the introduction, we now consider the setting in which the same key p is reused across different IVs. In AIM2, this amounts to changing the linear layer Lin. Let $L \in \mathbb{Z}_{>0}$ denote the number of IVs under which the key was (re)used, and denote by $\mathrm{Lin}_1, \ldots, \mathrm{Lin}_\mathrm{L}$ the corresponding linear layers. Then, the last equation of Proposition 1 is expanded into L equations (leading to $\ell + L$ base equations in total). Here, we are using the fact that $t_1, \ldots, t_\ell$ are all independent of the IV.

Proposition 2. *In the setting of Proposition 1 and multiple IVs providing Lin_1, …, Lin_L and the corresponding ciphertexts $c_1, \ldots, c_L$, the same statement holds with the polynomial $f_{\ell+1}$ replaced by L polynomials $f_{\ell+i}$, $i = 1, \ldots, L$:*

$$f_{\ell+i} = (p + c_i) \cdot \mathrm{Lin_i}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell) - \mathrm{Lin_i}(\mathrm{t}_1, \ldots, \mathrm{t}_\ell)^{2^{\mathrm{e}_*}}.$$

In the following subsection, we will show that the reused keys can also be replaced by specially related keys, under the same or different IVs.

2.3 Equivalent constants and related keys

We will first show that any instance of AIM2 can be manipulated to alter the constants by performing an affine substitution of input/output and a modification of the linear layer. This implies that one could set the first two constants to 0 and 1 respectively, without any change to security.

Proposition 3. *In an AIM2 instance, we can perform a change of variables p, c, constants, and the linear layer such that the new constants will satisfy $\gamma'_1 = 0, \gamma'_2 = 1$; the linear change of variable over p is given by $p' = (\gamma_1 + \gamma_2)^{-1}(p + \gamma_1)$; the new linear layer is given by multiplying its i-th input, $1 \leq i \leq \ell$, by $((\gamma_1 + \gamma_2)^{-1})^{-(2^{e_i}-1)^{-1}}$ and the output by $((\gamma_1 + \gamma_2)^{-1})^{(2^{e_*}-1)^{-1}}$; and the new ciphertext is $c' = (\gamma_1 + \gamma_2)^{-1}(c + \gamma_1)$.*

Proof. Let $a = (\gamma_1 + \gamma_2)^{-1}$. We are going to prove the correctness of this change of variables for the case $\ell = 2$. The other cases follow a similar proof.

Due to the change of variable and constants, the intermediate variables are $t'_i = a^{(2^{e_i}-1)^{-1}} t_i$. Furthermore, since

$$\begin{aligned} a^{-(2^{e_1}-1)^{-1}}(a(p + \gamma_1))^{(2^{e_1}-1)^{-1}} &= (p + \gamma_1)^{(2^{e_1}-1)^{-1}} \\ a^{-(2^{e_2}-1)^{-1}}(a(p + \gamma_1) + 1)^{(2^{e_2}-1)^{-1}} &= (p + \gamma_2)^{(2^{e_2}-1)^{-1}} \end{aligned}$$

The result after the linear layer is the same as the original one times $a^{(2^{e_*}-1)^{-1}}$. Thus, the new ciphertext satisfies

$$c' = (a^{(2^{e_*}-1)^{-1}}\mathrm{Lin}(\mathbf{t}))^{2^{e_*}-1} + \mathrm{p}' = \mathrm{a}(\mathrm{Lin}(\mathbf{t}) + \mathrm{p} + \gamma_1) = \mathrm{ac} + \mathrm{a}\gamma_1. \quad \square$$

From now on, we will consider $\gamma_1 = 0, \gamma_2 = 1$. We present a useful application of this result.

Proposition 4. *Let $\ell = 2, \gamma_1 = 0, \gamma_2 = 1$. The encryption process of a secret key $p_i = p^{2^i}$, $0 \leq i < n$, can be described by an application of an AIM instance with a modified linear layer to p. Therefore, any such corresponding ciphertext provides a new equation of the form (4), namely:*

$$f_{\ell+i} = (p + c_i^{2^{-i}}) \cdot \mathrm{Lin}(\mathrm{t}_1^{2^i}, \mathrm{t}_2^{2^i})^{2^{-i}} - \mathrm{Lin}(\mathrm{t}_1^{2^i}, \mathrm{t}_2^{2^i})^{2^{e_*-i}}.$$

When using Proposition 3, the new related keys p_i to be requested ($0 \le i < n$, $p_0 = p$) are expressed in terms of the original key p_{orig} as

$$p_i = \frac{(ap_{orig} + b)^{2^i} - b}{a}, \quad a = (\gamma_1 + \gamma_2)^{-1}, \quad b = \gamma_1(\gamma_1 + \gamma_2)^{-1}.$$

Proof. Given p_i, we define $t_{j,i}$ as the corresponding intermediate variable t_j for this key, and $u = \mathrm{Lin}(\mathrm{t}_{1,\mathrm{i}}, \mathrm{t}_{2,\mathrm{i}})$. We have:

$$\begin{aligned} t_{1,i} &= (p^{2^i})^{(2^{e_i}-1)^{-1}} = t_1^{2^i}, \\ t_{2,i} &= (p^{2^i} + 1)^{(2^{e_2}-1)^{-1}} = [(p+1)^{2^i}]^{(2^{e_2}-1)^{-1}} = t_2^{2^i}, \\ (u_i^{2^{-i}})^{2^{e_*}} &= (u_i^{2^{e_*}})^{2^{-i}} = (c_i + p^{2^i})^{2^{-i}} = c_i^{2^{-i}} + p. \end{aligned}$$

Thus, since the variables change in a predictable way and the operation of squaring can be moved into the linear layer, we can construct the equations for the new keys using the original variables. □

2.4 Mixed-order extended linearization (XL)

The basic linearization technique requires the number of linearly independent equations to be equal to the number of nonconstant monomials involved. When this does not hold, a technique called eXtended Linearization (XL) [CKPS00] can be used. The idea is to multiply the equations by all monomials of low degree, which increases the number of equations faster than the number of monomials, hoping that the linear independence is maintained. In another variant called XSL [CP02], only selected monomials are used in the multiplications.

For AIM, we observe that the equations in the basic model (Proposition 1) have low algebraic degree in the t_i variables (at most 1), but high degree over the $\mathbb{F}_{2^n}$ field. This prevents standard algebraic approaches such as those based on Gröbner basis. At the same time, the p variable has degree 1 in all equations. Thus, it is natural to consider all powers of 2 of p, which allows one to consider all powers of 2 of each equation. However, this approach is effectively equivalent to binarization, i.e., considering the equations over $\mathbb{F}_2$, as illustrated by the following standard statement (for completeness, the proof is given in Section E).

Proposition 5. *Let $y \in \mathbb{F}_{2^n}$ be a variable. Then, there exist n variables $\hat{y}_0, \dots, \hat{y}_{n-1} \in \mathbb{F}_2$ and an $\mathbb{F}_{2^n}$-linear bijection between vectors*

$$(y, y^2, y^4, y^8, \dots, y^{2^{n-1}}) \quad \text{and} \quad (\hat{y}_0, \dots, \hat{y}_{n-1}).$$

We propose the idea of a *mixed ordering*, which is a heuristic consisting of ordering the monomials in t_i variables by the algebraic degree (i.e. the Hamming weight of the exponents), while ordering the monomials in p by the usual degree. This guides the possible choices of monomials for the multiplications. According to Proposition 5, this means that we can perform an $\mathbb{F}_{2^n}$-linear variable change to variable $\hat{t}_{i,j} \in \mathbb{F}_2$, $1 \le i \le \ell$, $0 \le j < n$, while working at the same time with full $p \in \mathbb{F}_{2^n}$. Although the described change of basis provides more insight into the mixed ordering idea (as well as a direction to reinterpret the attacks within the Gröbner basis framework), it is not necessary to be explicitly performed for our attacks, and we will usually linearize monomials $t_i^{2^j}$ directly.

Definition 1 ((M, L)-system). *For parameters $M \in \mathbb{Z}_{\ge 0}, L \in \mathbb{Z}_{>0}$, define an (M, L)-system as the set of polynomials*

$$\begin{aligned} S_{M,L} = \left\{ f_i^{2^m} \cdot p^j \right\}_{i,m,j} &\subseteq \mathbb{F}[t_1, \dots, t_\ell, p], \\ i \in \{1, \dots, \ell + L\}, \quad m \in \{0, \dots, M\}, \quad j &\in \{0, \dots, 2^M - 2^m\}, \end{aligned}$$

where f_i are defined as in Proposition 1 and Proposition 2 (this system corresponds to L different IVs/related-keys).

While this basic definition is sufficient for our main attacks, we will later extend it by allowing multiplications by $t_i^{2^u}$ monomials, in line with the mixed-order heuristic.

2.5 Simple and practical reused-key and related-key attacks

We propose an attack based on the construction of an (M, L)-system with enough equations so that we can apply traditional linearization by converting each monomial into a new variable. Specifically, we want to find an equation of form $p = a$, where a is some constant that coincides with the real value of the private key. To obtain the number of equations needed, we can ask for new linear layers to obtain equations of the form (4), or perform a related-key attack as explained in Proposition 4. Table 3 presents the estimated time complexity (and specific time taken for fast instances) for different values of M, L. In every case, we obtain maximum rank.

The time complexity is estimated by using the Wiedemann algorithm [Wie86], which has complexity $\mathcal{O}(N^2 s)$, where N is the dimension of the matrix and s is the average number of nonzero entries per row. In the attack, the complexity is equal to $\mathcal{O}(\ell^3 n^3 2^{2M})$, due to the number of columns of the matrix being $(\ell n+1)(2^M+1)$, and the number of entries per row at most $2\ell n$ (although the first ℓ equations and their squarings are much more sparse). In addition, we provide some basic analysis of the rank of the system in Section D.

n	L	M	Time complexity
128	255	1	$2^{27.19}$/2min
128	109	3	$2^{30.36}$/23min
128	2	66	$2^{156.02}$
128	1	87	$2^{198.02}$

n	L	M	Time complexity
192	383	1	$2^{28.94}$/7min
192	163	3	$2^{32.11}$/80min
192	2	98	$2^{221.77}$
192	1	130	$2^{285.77}$

n	L	M	Time complexity
256	766	1	$2^{31.93}$/50min
256	327	3	$2^{35.1}$/11h
256	2	155	$2^{338.76}$
256	1	194	$2^{416.76}$

Table 3: Parameters of the attacks by linearization. The complexity is given in field operations.

3 New elimination technique using partial linearization and polynomial matrix algorithms

In this section, we present a new general elimination framework. We consider an arbitrary field $\mathbb{F}$, variables p and $\mathbf{t} = (t_1, \ldots, t_\ell)$, and U a set of monomials in $\mathbf{t}$.

The core idea is to perform *partial* XL multiplication and linearization - on all $\mathbf{t}$ variables. Subsequently, we work with vectors of *univariate polynomials in* p, eliminating coordinates related to $\mathbf{t}$ variables to obtain a univariate polynomial in p from which we can recover roots in $\mathbb{F}$ using standard techniques in quasilinear (in the degree) time. An alternative viewpoint is that we treat p as a scalar which parameterizes the linearized system of equations, and we perform linear algebra over the corresponding $\mathbb{F}[p]$-module.

3.1 Polynomial matrix formulation of elimination

We start by defining a representation of polynomials as $\mathbb{F}[p]$-vectors. An interpretation is that we linearize all the $\mathbf{t}$-monomials and forget the algebraic relations between these variables; at the same time, we treat the polynomials in p as scalars. From now on, we will assume a fixed, finite, ordered set $U \subseteq \mathbb{Z}^\ell_{\geq 0}$ such that $(\mathbf{0} \in \mathbb{Z}^\ell) \in U$, and we denote $W = |U|$. Essentially, U corresponds to the set of linearized monomials in the usual XL method, except that we ignore one variable p, which reduces this set size.

Definition 2. *For a polynomial $f \in \mathbb{F}[\mathbf{t}, p]$ such that f can be written as*

$$f = \sum_{u \in U} \alpha_u(p) \mathbf{t}^u, \qquad \alpha_u \in \mathbb{F}[p], \tag{5}$$

define the p-form *of f, denoted by $f|_p$, as the vector*

$$f|_p = (\alpha_u(p) \mid u \in U) \quad \in \mathbb{F}[p]^W.$$

Note that the p-form is just a representation, and f can be fully recovered from $f|_p$ (knowing U) using (5). The following proposition shows that the set of all p-forms (for a fixed U) can be viewed as an $\mathbb{F}[p]$-module.

Proposition 6. *For any $f \in \mathbb{F}[\mathbf{t}, p]$, multiplication of f by any $g \in \mathbb{F}[p]$ acts on $f|_p$ coordinate-wise:*

$$(f \cdot g)|_p = f|_p \cdot g(p) = (\alpha_u(p) \cdot g(p) \mid u \in U).$$

The set of all p-forms with respect to U forms a free $\mathbb{F}[p]$-module, with addition defined coordinate-wise, and scalar multiplication by $g \in \mathbb{F}[p]$ defined as above.

Proof. The statement trivially follows from the definitions; the freeness of the module is proven by the standard basis $\{(1, 0, \ldots, 0), (0, 1, 0, \ldots, 0), \ldots\}$. □

Definition 3. *For a set $F \subseteq \mathbb{F}[\mathbf{t}, p]$ of polynomials, let $\mathbf{F} \in \mathbb{F}[p]^{H \times W}$ be a polynomial matrix whose rows are the vectors $f|_p$ for all $f \in F$ in any order, where $H = |F|$ and $W = |U|$. Furthermore, let $\mathbf{F}_+ \in \mathbb{F}[p]^{H \times (W-1)}$ be the matrix $\mathbf{F}$ with the column corresponding to $\mathbf{0} \in U$ removed.*

With this notation, the problem of eliminating all $\mathbf{t}$ variables can be formulated as finding a non-trivial vector from the left kernel of the polynomial matrix $\mathbf{F}_+$.

Proposition 7. *Let $F = \{f_1, \ldots, f_H\} \subseteq \mathbb{F}[\mathbf{t}, p]$. Then, there exists a one-to-one correspondence between $\mathbb{F}[p]$-combinations of f_i belonging to $\mathbb{F}[p]$ (that is, eliminating $\mathbf{t}$), and the left kernel of the matrix $\mathbf{F}_+$.*

Proof. Let $(z_1, \ldots, z_H)$ be such a combination. Then we can multiply it to the matrix $\mathbf{F}_+$, and the fact that the combination belongs to $\mathbb{F}[p]$ implies that the multiplication must be zero (recall that each column represents one monomial over $\mathbf{t}$). An analogous argument proves the inverse implication. □

Theorem 1. *If $\mathbf{F}$ has full column-rank, then there exists an $\mathbb{F}[p]$-combination of f_i belonging to $\mathbb{F}[p] \setminus \{0\}$.*

Proof. Apply division-free elimination to $\mathbf{F}$. The correspondent sequence of multiplications of rows generates a $\mathbb{F}[p]$-combination of f_i (corresponding to the reduction of the last row). In this combination, only the last column (of the last row) can be nonzero. Furthermore, it has to be nonzero (otherwise, the matrix would not have full rank). Thus, the combination belongs to $\mathbb{F}[p] \setminus \{0\}$. □

Remark 1. The converse is generally not true. There may exist nontrivial nonzero $\mathbb{F}[p]$-combinations of given polynomials eliminating $\mathbf{t}$ without F being full column-rank. An obvious example is $F = (t_1 + t_2, p)$, where p clearly belongs to $\mathbb{F}[p]$, but F is not full column-rank, since $(0, 0), (0, 1), (1, 0) \in U$ while $H = 2$. However, it is difficult to prove the existence of such linear combinations, even if $H > W$, since all left kernels of the truncated matrix can lead to $0 \in \mathbb{F}[p]$. In fact, this often happens in practice in algebraic attacks due to redundant multiplications, which are difficult to manage or predict.

Remark 2. This statement is similar to [BNSED22, Lemma 3.4], which however works in a special setting (Gröbner basis in shape position and stability assumption) where the converse does hold.

Calculating the rank of a polynomial matrix is a complicated task. However, there is a strong lower bounding method based on evaluating the matrix at any fixed p. This includes $p = 0$, so we can lower bound the rank of a matrix by computing the rank of the matrix of constant coefficients of the respective polynomials, which is feasible if the dimensions of the matrix are not too large.

Proposition 8. *Let $\mathbf{F} = \mathbf{F}(p) \in \mathbb{F}[p]^{H \times W}$. Then, for all $p_0 \in \mathbb{F}$, $\operatorname{rank} \mathbf{F} \geq \operatorname{rank} \mathbf{F}(p_0)$.*

Proof. Suppose that $\operatorname{rank}\mathbf{F}(p_0) > \operatorname{rank}\mathbf{F}$. Then, there is a minor of $\mathbf{F}$ whose determinant is 0, while the same minor in $\mathbf{F}(p_0)$ has a nonzero determinant. This is not possible, since 0 cannot evaluate to nonzero at any $p = p_0$. □

If $p = 0$ fails to achieve full rank (which depends on the equation system and cannot be bounded generally; for example, if $p = 0$ is always a spurious uninteresting solution), we can evaluate the matrix at any random p and then compute the rank, which should have high success probability (and we can use a field extension).

Proposition 9. *For all but a finite number of p_0 in $\overline{\mathbb{F}}$, it holds* $\operatorname{rank}\mathbf{F} = \operatorname{rank}\mathbf{F}(p_0)$.

Proof. Given a minor of $\mathbf{F}$, its determinant corresponds to a polynomial. The corresponding minor's determinant of $\mathbf{F}(q)$, for any $q \in \overline{\mathbb{F}}$, corresponds to the evaluation of the polynomial on q. Thus, if a determinant of a minor of $\mathbf{F}$ is nonzero, but the determinant of the corresponding minor of $\mathbf{F}(q)$ is zero, then q has to be a root of the polynomial that generates the determinant.

Since there is a finite number of minors, each one of them generating a finite degree polynomial, and each polynomial has at most as many roots as its degree, there is only a finite number of elements q of $\overline{\mathbb{F}}$ such that the rank of $\mathbf{F}(q)$ is not the same as the rank of $\mathbf{F}$. □

Toy example We propose the following toy example so that it is easier to understand how this new model works. Let us say that we have the following system of equations on three variables over $\mathbb{F}$:

$$\begin{aligned} t_1 + t_2 &= 0, \\ t_1 + p \cdot t_2 &= 0, \\ t_2 \cdot p^3 + p^2 &= 0. \end{aligned}$$

We can create the matrix $\mathbf{F}$ over $\mathbb{F}[p]$ using the first column for the component of t_1, the second column for the component of t_2, and the last column for the constant component. Reducing the matrix without divisions leads to a new matrix $\mathbf{F}'$:

$$\mathbf{F} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & p & 0 \\ 0 & p^3 & p^2 \end{pmatrix}, \qquad \mathbf{F}' = \begin{pmatrix} 1 & 1 & 0 \\ 0 & p-1 & 0 \\ 0 & 0 & p^2 \cdot (p-1) \end{pmatrix}$$

The last component of the matrix turns out to be a degree-3 polynomial over p with solutions 0 and 1. It can be easily verified that they coincide with solutions to the original system of equations.

3.2 State-of-the-art algorithms for kernel and order bases computation

There are multiple works that proposed various efficient algorithms for the problem of computing the kernel of a polynomial matrix based on the so-called order bases [GJV03,ZLS12,JNV20].

Throughout this section, we assume a polynomial matrix $\mathbf{G} \in \mathbb{F}[p]^{H \times W}$ over a field $\mathbb{F}$. Let ξ_i, $1 \leq i \leq H$, denote the i-th row-degree (maximum polynomial degree in the row), and let $\xi = \sum_{i=1}^{H} \xi_i$ denote their sum.

First, we will introduce the main relevant definitions[5].

Definition 4 (Kernel/approximant basis). *Let $\mathbf{G} \in \mathbb{F}[p]^{H \times W}$. A* kernel basis *is an (ordered) basis of the left kernel of $\mathbf{G}$, that is, of the $\mathbb{F}[p]$-module*

$$\left\{ f \in \mathbb{F}[p]^H \mid f \times \mathbf{G} = 0 \right\}.$$

[5] We adopt the row-wise notation with a focus on the left kernel of the matrix, which is dominant across the cited works, excluding only [ZLS12].

It is said to be minimal *if the vector of its degrees is lexicographically minimal among all such bases.*

Let $o \in \mathbb{Z}_{>0}$. *An* approximant basis *of order* o *of* $\mathbf{G}$ *is a basis of the* $\mathbb{F}[p]$*-module*

$$\left\{ f \in \mathbb{F}[p]^H \mid f \times \mathbf{G} \equiv 0 \pmod{p^o} \right\}.$$

It is also called an order basis *of order* o *if the vector of degrees of the basis vectors is lexicographically minimal among all such bases.*

A widely used method for computing the kernel basis is to compute an order basis of a sufficiently large order which would yield the kernel basis. Generally, $o = W + dW$ is sufficient, where d is the upper bound on the polynomial degree of $\mathbf{G}$. However, a more precise bound is possible.

Theorem 2 ([ZLS12, Thm 3.4]). *Let* $\mathbf{G} \in \mathbb{F}[p]^{H \times W}$. *Then, the sum of the* ξ_i*-shifted degrees of the minimal kernel basis of* $\mathbf{G}$ *is upper bounded by* ξ*, which means*

$$\sum_z \max_i (\xi_i + \deg z_i) \leq \xi,$$

where the sum is over the minimal kernel basis vectors z.

Corollary 1. *If* $\mathbf{G}$ *has full rank* W *and* $H = W + 1$*, the minimal kernel basis of* $\mathbf{G}$ *is generated by a single vector* $z \in \mathbb{F}[p]^H$ *whose coordinates have degrees upper bounded as*

$$\deg z_i \leq \xi - \xi_i.$$

The work [ZLS12] proposes a more advanced technique that computes an order basis of lower order and then gradually transforms it into the kernel basis, while maintaining a specific degree bound. The authors claim in [ZLS12, Thm.4.1] complexity $\tilde{\mathcal{O}}(W^{\omega-1}\xi)$ when $H = \mathcal{O}(W)$, $\xi \geq W$. Unfortunately, the authors do not provide a more precise analysis of their method: the $\tilde{\mathcal{O}}$-notation hides an unclear number of logarithmic factors. The first step in [ZLS12, Algorithm 1] is the calculation of the order basis of order $3s$, where s is the average degree of the W largest-degree rows. Using the recent algorithm from [JNV20] (or the one directly cited [GJV03]), it requires at least $\mathcal{O}(W^\omega \mathbf{M}(s) \log s) = \mathcal{O}(W^\omega s \log^2 s \log\log s)$ field operations.

3.3 A new method tailored to our attack

In this section, we present a new method for finding a *single* kernel vector of a *sparse* polynomial matrix, which would perfectly fit our application to AIM2. The high-level idea of the algorithm is to first compute an arbitrary approximant basis of the matrix (not necessarily minimal) for a well-chosen order o. This basis will consist of one single vector $z \in \mathbb{F}[p]^H$. Then, we will use the HalfGCD algorithm (see e.g. [GG13, Thm.9.6], [vdH25]) to find a polynomial $q(p) \in \mathbb{F}[p]$ that minimizes the degrees of $q(p) \cdot z \mod p^o$. More precisely, we will focus on one entry of z and choose the approximation order double of its degree bound in the minimal kernel. Then, HalfGCD of this entry of z with p^o should produce the desired polynomial q.

Step 1: Approximant vector computation The first step (approximant vector computation) is implemented efficiently due to the assumed sparsity of the matrix and restriction to the case of a single kernel vector. It is provided in Algorithm 1. The algorithm constructs the approximant vector in a simple iterative way, modifying the solution in such a way that, at each step d, the new vector z satisfies $z \times G \equiv 0 \pmod{p^{d+1}}$.

In general, it is quite similar to [GJV03, SigmaBasis], which generally requires time quadratic in the polynomial degree d of the matrix, and is used in that paper only as a building block with $d = 1$. Our insight is that the intermediate computation with quadratic complexity can be more efficient for coefficient-sparse matrices; furthermore, we simplify the algorithm for the case of full-rank matrix with $H = W + 1$. It is also much easier to implement and analyze.

Algorithm 1 Approximant vector of a polynomial matrix

Inputs: matrix $G \in \mathbb{F}[p]^{H \times W}$ with s nonzero coefficients in total, $H > W$, $\operatorname{rank} G(0) = W$, order $o \in \mathbb{Z}_{>0}$
Output: nonzero vector $z \in \mathbb{F}[p]^H_{<o}$ such that $z \times G \equiv 0 \pmod{p^o}$

1: $G_0 \leftarrow G(0) \quad \in \mathbb{F}^{H \times W}$ ▷ $G(0) = G \mod p$
2: $z \leftarrow$ any nonzero left kernel vector of G_0, $\quad z \in \mathbb{F}^H$ ▷ $z \times G_0 = 0$
3: **for** $d \in \{1, \ldots, o-1\}$ **do**
4: $\quad v \leftarrow ((z \times G) \cdot p^{-d}) \mod p, \quad v \in \mathbb{F}^W$ ▷ $\mathcal{O}(s)$
5: $\quad t \leftarrow$ any nonzero solution to $t \times G_0 = v, \ t \in \mathbb{F}^H$ ▷ $\mathcal{O}(W^2)$ with precomp.
6: $\quad z \leftarrow z - t \cdot p^d$ ▷ Extend in-place, $\mathcal{O}(W)$
7: **return** z

Remark 3. In lines 2 and 5, if we fix a single non-pivot of G_0 to always use, the corresponding coordinate of z in the output will be 1. In this way, we only need to perform a matrix-vector multiplication in line 5 at each iteration.

Remark 4. In the algorithm, all steps are based on basic field operations. In particular, we avoid FFT and recursion. Line 2 requires basic linear algebra over the ground field, but this step is typically negligible and can be implemented naively.

Theorem 3. *Algorithm 1 is correct and requires* $\mathcal{O}(W^{\omega-1}H)+ \sim 2(W^2 + s) \cdot o$ *field operations.*

Proof. (Correctness) We will show that, at every iteration d of the loop, z satisfies $z \times G \equiv 0 \pmod{p^{d+1}}$. First, in line 2, $z \times G_0 = z \times G(0) = z \times G \mod p^1$. Now, let $d \in \{1, \ldots, o-1\}$ such that z obtained in the previous iteration of the loop satisfies $z \times G \equiv 0 \pmod{p^d}$. From the expression of v in line 4, we conclude that

$$z \times G = v \cdot p^d + R \cdot p^{d+1}$$

for some $R \in \mathbb{F}[p]^W$. Let us call z' the new z obtained in iteration d. The vector t from line 5 satisfies

$$t \times G = v + p \cdot R'$$

for some $R' \in \mathbb{F}[p]^W$. We have

$$\begin{aligned} z' \times G &= z \times G - (t \times G) \cdot p^d = z \times G - v \cdot p^d - R' \cdot p^{d+1} \\ &= R \cdot p^{d+1} - R' \cdot p^{d+1} \equiv 0 \mod p^{d+1}. \end{aligned}$$

Thus, at the end of the for loop, $z \times G \equiv 0 \mod p^o$ and z is nonzero since, after initial assignment at line 2, its value modulo p remains unchanged.

(Complexity) Line 1 only considers the constant coefficients of the polynomials (no computations). Line 2 requires performing linear algebra on G_0, which can be done by working on square blocks of size W, leading to complexity $\mathcal{O}(W^\omega(H/W)) = \mathcal{O}(W^{\omega-1}H)$. In particular, this time allows us to find the W pivots and invert the corresponding submatrix. Lines 3-6 of the algorithm constitute a simple loop repeated $o-1$ times. Line 4 computes the degree-d "slice" of coefficients of the product $z \times G$. For each nonzero coefficient of G, there is at most one monomial in the corresponding coordinate of z which leads to a degree-d monomial. Thus, it costs at most 1 multiplication and 1 addition per nonzero coefficient of G, $2s$ in total per loop iteration. Line 5 can be implemented by a matrix-vector multiplication (inverted submatrix on the pivot rows), which requires at most $2W^2$ operations (1 multiplication and 1 addition per matrix entry). Line 6 simply writes down the previous result as new top coefficients of z and incurs no cost. We obtain the claimed cost. □

Proposition 10. *If* $H = W + 1$*, then the output vector* z *of Algorithm 1 generates all such vectors modulo* p^o*, that is, any other approximant vector of order* o *of* G *is an* $\mathbb{F}[p]$*-multiple of* z*.*

Proof. First, we are going to prove that the degree-0 coefficient of at least one coordinate z (say, z_i) is nonzero. In line 2, by definition, it is a nonzero vector of constants. Now, let us suppose that the

output z after iteration d has this property. Then, letting z' be the output after iteration $d+1$, we have $z' = z - t \cdot p^{d+1}$, where t is a vector of constants; thus, the same coordinate has to still be non-zero in its constant coefficient.

Once we know this, it is easy to show that any other approximant vector of order o of G is an $\mathbb{F}[p]$-multiple of z. The number of such approximants is $|\mathbb{F}|^o$, which coincides with the cardinality of the set of $\mathbb{F}[p]$-multiples of z modulo p^o, counting repetitions. Let us see that there are no repetitions.

Suppose that there exist two polynomials q, r in $\mathbb{F}[p]$, of degree less than o, such that $z \cdot q \equiv z \cdot r$ and thus, $z_i \cdot q \equiv z_i \cdot r$. This means that $z_i \cdot (q - r)$ is a multiple of p^o. Since z_i has a constant term, it follows that $q \equiv r$ modulo p^o. □

Step 2: Kernel vector computation (degree minimization) Let $G \in \mathbb{F}[p]^{(W+1)\times W}$ (that is, $H = W + 1$) with $\operatorname{rank}(G(0)) = W$. According to Corollary 1, there must exist a minimal kernel vector $z \in \mathbb{F}[p]^H$ that generates the full kernel of G, with degree bounded by $\deg z_i \le \xi - \xi_i$, where ξ_i is the degree of the i-th row of G and ξ is their sum. We will compute an approximant basis (one vector) $\overline{z}$ of order $o = 2\xi + 2$ using Algorithm 1 (Proposition 10 guarantees that the output vector forms an approximant basis of order o). There must exist a polynomial $q(p) \in \mathbb{F}[p]_{<o}$ such that $q(p) \cdot \overline{z} \equiv z \pmod{p^o}$. Thus, we will find a polynomial $q'(p)$ for which $z' = q'(p) \cdot z \mod p^o$ satisfies the degree bound and show that z' is equal to z up to a constant multiple, which is ensured due to the sufficiently large approximation order o.

Algorithm 2 describes a probabilistic version of the method, where we find the required multiple by a single HalfGCD with a random linear combination of z (in practice, we just choose one coordinate arbitrarily). We also note that, heuristically, computing line 7 is usually not necessary. For the general interest, a deterministic version of Algorithm 2 is described in Section B. It requires a sequence of HalfGCD calls instead of one, and is thus more expensive.

Algorithm 2 Minimal kernel vector of a polynomial matrix of nullity one

Inputs: matrix $G \in \mathbb{F}[p]^{(W+1)\times W}$ with s nonzero coefficients in total, $\operatorname{rank} G(0) = W$
Output: vector $z \in \mathbb{F}[p]^{W+1}_{\le\xi}$ such that $z \times G = 0 \in \mathbb{F}[p]^W$ and $\xi = \sum_{i=1}^{W+1} \deg G_i$, where G_i is the i-th row of G

1: $o \leftarrow 2\xi + 2$
2: $\overline{z} \leftarrow \textsc{ApproximantVector}(G, o)$ ▷ Algorithm 1
3: assume w.l.o.g. $p \nmid z_1(p)$
4: $\hat{z} \leftarrow \overline{z}/\overline{z}_1^{-1} \mod p^o$ ▷ Can be skipped, Remark 3 ensures a coordinate equal to 1
5: $t \leftarrow \sum_{i=2}^{W+1} \alpha_i \cdot \hat{z}_i, \quad \in \mathbb{F}[p],$ ▷ Each $\alpha_i \leftarrow \mathbb{F}$ is sampled uniformly at random
6: $(u, w, r) \leftarrow \text{HalfGCD}(t, p^o)$ ▷ $u \cdot t + w \cdot p^o = r, \deg u \le o/2, \deg r < o/2$
7: $u' \leftarrow u/\text{GCD}(u, r)$
8: **return** $z = u' \cdot \hat{z}$

In contrast to Algorithm 1, this step requires polynomial arithmetic (multiplication, division, HalfGCD), which requires FFT operations. In our experiments, we used the implementation of the Frobenius Additive FFT [LCK+18].

We will need the following lemma for showing uniqueness of rational approximation. This construction is an analogue of Padé approximation for finite fields.

Lemma 1. *Let d be a positive integer, let $f(p), g(p) \in \mathbb{F}[p]_{\le d}$ and define $w(p) = \gcd(f(p), g(p))$. Then, for any $r(p) \in \mathbb{F}[p]_{<2d}$, if both $F(p) = r(p)f(p) \mod p^{2d}$ and $G(p) = r(p)g(p) \mod p^{2d}$ have degrees less than d, then $F(p)$ is a polynomial multiple of $f'(p) = f(p)/w(p)$ and $G(p)$ is a polynomial multiple of $g'(p) = g(p)/w(p)$.*

Proof. We have

$$g(p)F(p) \equiv r(p)f(p)g(p) \equiv f(p)G(p) \pmod{p^{2d}}.$$

Since both $g(p)F(p)$ and $f(p)G(p)$ have degree at most $2d - 1$, they are equal over $\mathbb{F}[p]$. We have

$$f(p)G(p) = w(p)f'(p)G(p) = w(p)g'(p)F(p) = g(p)F(p)$$

and thus $f'(p)G(p) = g'(p)F(p)$. Since f' and g' are coprime, the statement follows. □

Theorem 4. *Algorithm 2 succeeds with probability at least* $1 - \xi/|\mathbb{F}|$ *(over the choice of random combination in line 5).*

Proof. Let $v = (v_1, \ldots, v_{N+1}) \in \mathbb{F}[p]$ be the unique vector (up to $\mathbb{F}$ -multiplication) generating the left kernel of G (we are sure of both its existence and uniqueness by the rank-nullity theorem). By Proposition 10, the vector $\overline{z}$ in the algorithm is an $\mathbb{F}[p]$-multiple of v modulo p^o. Note that the factor has to be coprime to p, otherwise $\overline{z}$ would not be a generator of all approximant vectors of order o. Without loss of generality, we assume that $p \nmid v_1$ and, by the previous consideration, $p \nmid \overline{z}_1$. Then, we have $z = (1, z_2, \ldots, z_{W+1})$ being an $\mathbb{F}[p]$-multiple of v modulo p^o by an invertible factor (namely $v_1^{-1} \bmod p^o$). In particular, we have

$$t \equiv \sum_{i=2}^{W+1} \alpha_i \cdot \frac{v_i}{v_1} \equiv \frac{\sum_{i=2}^{W+1} \alpha_i \cdot v_i}{v_1} \pmod{p^o}. \tag{6}$$

The outputs u, w, r from HalfGCD satisfy $u \cdot t \equiv r \pmod{p^o}$ where u has degree at most $o/2$, r has degree lower than $o/2$. By Lemma 1, $u' = u/\mathrm{GCD}(u, r)$ and $r' = r/\mathrm{GCD}(u, r)$ are uniquely defined; they also satisfy $u't \equiv r' \pmod{p^o}$ and are not divisible by p. Therefore, we can write $t \equiv \frac{r'}{u'} \pmod{p^o}$, which is a canonical form of t up to $\mathbb{F}$-multiplication of r' and u' (which means, there is no common factor among numerator and denominator). We now prove that (6) is also canonical with high probability, and thus u' is equal to v_1 up to $\mathbb{F}$-multiplication.

First, observe that there cannot be a common factor of $v_1, \ldots, v_{W+1}$ of degree 1 or higher (otherwise, v would not be a kernel generator). Now, consider a root y of v_1 over the algebraic closure. By the above discussion, at least one of v_j, $2 \leq j \leq W+1$, does not have y as root (otherwise there would be a common factor of all v_i). For each root y_i of v_1, define a linear function $\gamma_i : (\alpha_2, \ldots, \alpha_{W+1}) \mapsto \sum_{i=2}^{W+1} \alpha_i v_i(y_i)$ and the product $\gamma(\alpha_2, \ldots, \alpha_{W+1}) = \prod_i \gamma_i$, which is nonzero and has degree at most ξ - the maximum number of roots of v_1 over the algebraic closure of $\mathbb{F}$ (again, the roots are fixed and the variables are all the α_i). By the Schwartz-Zippel lemma, γ is equal to zero in at most $\xi/|\mathbb{F}|$ fraction of all possible values of $\alpha_2, \ldots, \alpha_{W+1}$ (which is maximized when we consider the base field and not an extension). Therefore, for the remaining values, the numerator $\sum_{i=2}^{W+2} \alpha_i \cdot v_i$ is evaluated as nonzero at all roots y_i of v_1 and, thus, has no common factors with it. We conclude that u' is equal to v_1 (up to $\mathbb{F}$-multiple) and, therefore, $u' \cdot z$ recovers v, the kernel generator of G. □

Proposition 11. *One call to Algorithm 2 requires*

$$\sim 4\xi(W^2 + s) + \mathbf{M}(\xi)(11 \log_2 \xi + 2W) \;=\; \mathcal{O}(\xi(W^2 + s) + \mathbf{M}(\xi)(W + \log_2 \xi))$$

operations, under the FFT model (general case) from [vdH25].

Proof. Line 2 together with Theorem 3 proves the first two terms (note $o = 2\xi + 2$ adds an extra factor of 2). Line 4 is not needed due to Remark 3. Line 5 requires $\sim W$ scalar multiplications of polynomials of degree at most $2\xi + 2$ and their sum, which is in $\sim 4W\xi$. Line 6 requires $\sim 2.75\mathbf{M}(2\xi) \log_2 2\xi$ operations according to GCD cost analysis (see Proposition 17). Line 7 can reuse the HalfGCD computation from 6; with two multiplications (time $2\mathbf{M}(2\xi)$) the GCD problem is reduced to a half-sized one. Thus, the GCD can be computed in at most twice the time of HalfGCD (in the $\sim$ notation). Polynomial division can be done in time $\mathcal{O}(\mathbf{M}(2\xi))$. We conclude that lines 6-7 together cost $\sim 5.5\mathbf{M}(2\xi) \log_2 2\xi \sim 11\mathbf{M}(\xi) \log_2 \xi$ field operations. Line 8 computes the W products of two polynomials of degree at most $2\xi + 2$, in time $W \cdot \mathbf{M}(2\xi) \sim 2W \cdot \mathbf{M}(\xi)$. Writing down the dominant terms, we get the claim. □

3.4 Univariate root finding and final complexity analysis

Applying techniques from the previous subsection to the truncated basis matrix $\mathbf{F}_+$ from Subsection 3.1, we obtain a vector $z \in \mathbb{F}[p]^H$ being a nontrivial kernel of $\mathbf{F}_+$: $z \times \mathbf{F}_+ = 0$. If $\mathbf{F}$ has full rank, this yields a non-trivial univariate polynomial in the ideal of the corresponding system (see Theorem 1): $z \times \mathbf{F} = (g(p), 0, \ldots, 0,)$, $g(p) \neq 0$. The following proposition upper-bounds the degree of g. We will use ξ_+ and ξ respectively to denote the sum of row-degrees of $\mathbf{F}_+$ and $\mathbf{F}$ respectively.

Proposition 12. *If z is a minimal kernel vector of $\mathbf{F}_+$, then for $g = (z \times \mathbf{F})_1$, we have*

$$\deg g \;\leq\; \xi' = \xi_+ + \max_{1\leq i\leq H}(\deg \mathbf{F}_{i,1} - \deg \mathbf{F}_{+,i}) \;\leq\; \xi,$$

where $\mathbf{F}_{+,i}$ is the i-th row of $\mathbf{F}_+$, and $\mathbf{F}_{i,1}$ is the i-coordinate of the column excluded from $\mathbf{F}$ to get $\mathbf{F}_+$.

Proof. From Corollary 1 we know that $\deg z_i$ is upper-bounded by $\xi_+ - \deg \mathbf{F}_{+,i}$:

$$\deg g(p) \leq \max_{1\leq i\leq H}(\deg z_i + \deg \mathbf{F}_{1,i}) \leq \max_{1\leq i\leq H}(\xi_+ - \deg \mathbf{F}_{+,i} + \deg \mathbf{F}_{i,1}).$$

For i achieving the maximum in the middle expression, the expression is equal to ξ_+ where one row-degree of $\mathbf{F}_+$ is replaced by the corresponding row-degree of $\mathbf{F}$, which cannot be greater than ξ. □

As usual in such applications [BBLP22], it is reasonable to assume that the resulting polynomial only has a small number of roots in the target field. Then, a standard cost analysis (see Proposition 18) gives us the cost of $\mathbf{M}(\xi')(2n + 11\log_2 \xi')$ field operations for finding these roots, in the case of the binary field.

Note that this cost is comparable to that of finding the kernel vector and computing $g(p)$, when ξ_+ and ξ' are close. In our applications, we will have $\xi_+ = \xi' = \xi$ and from now on we simply use ξ (which would still be correct would they differ, since it is an upper bound). The following proposition summarizes the cost of the full attack.

Theorem 5. *Under stated assumptions (full-rank matrix $\mathbf{F}(0)$, small number of roots of g in $\mathbb{F}$), in the binary field case $\mathbb{F} = \mathbb{F}_{2^n}$, the cost of finding candidates for p in $\mathbb{F}$ is given by*

$$\sim \xi \cdot (4W^2 + 6s) \quad + \quad \mathbf{M}(\xi) \cdot (2n + 2W + 22\log_2 \xi)$$

field operations, where ξ is the row-degree sum of $\mathbf{F}$. The memory requirement is $\sim 2\xi W + \mathcal{O}(s)$ field elements.

Proof. The time complexity follows the previous derivation, consisting of: Algorithm 2 (Proposition 11), computing $z \cdot \mathbf{F}_0$ (naively in in $\sim 2\xi \cdot s$ – one multiplication and one addition per nonzero coefficient[6]), computing univariate roots (Proposition 18). Regarding memory complexity, note that the equation matrix is sparse and only requires $\mathcal{O}(s)$ memory for storing. The approximant vector is dense and thus requires $o \cdot (W + 1) = (2\xi + 2)(W + 1) \sim 2\xi W$ elements. Further HalfGCD and GCD computations work with single degree-o polynomials, adding negligible cost. □

The theorem directly generalizes to non-binary finite fields $\mathbb{F}_q$ with the same $\mathcal{O}$-asymptotic (where $n = \log_2 q$), and concrete coefficients depending on the arithmetic available/selected (FFT/NTT, Schönhage-Strassen, etc.). We give the following high-level interpretation of the result.

For a sparse system of equations with high degrees in $\mathbf{t}$-monomials (which implies a large number W of such monomials), the complexity is dominated by the factor $\mathcal{O}(\xi \cdot W^2)$. Here, "high" refers to comparison with a poly-logarithm of the degree in p, meaning that this case is the most typical.

For a sparse system with low degrees in $\mathbf{t}$-monomials / low number of such monomials, the complexity is essentially a small multiple of univariate root finding of a degree-ξ polynomial.

3.5 Relation to the determinant

In this subsection, we uncover a close relationship between our elimination polynomial $g(p)$ and the determinant of the initial equation matrix $\mathbf{F}$. We recall that the adjugate matrix $\mathrm{Adj}(\mathbf{F})$ of $\mathbf{F}$ is defined as the transpose of the cofactor matrix of $\mathbf{F}$, which in turn consists of properly signed minors of $\mathbf{F}$ (in particular, it always exists). It is such that $\mathrm{Adj}(\mathbf{F}) \times \mathbf{F} = \det(\mathbf{F}) \times \mathrm{Id}_W$, where Id_W is the identity matrix of order W (note that $\det(\mathbf{F}) \in \mathbb{F}[p]$).

[6] In practice, the nonzero coefficients are distributed evenly across the columns of $\mathbf{F}$, and thus this term becomes $2\xi/W \cdot s$ and can be ignored. In our complexity estimations, we will use $\xi \cdot (4W^2 + 4s)$ for the first part.

Proposition 13. *Let* $\mathbf{F} = \mathbf{F}(p) \in \mathbb{F}[p]^{W \times W}$ *and* $z \in \mathbb{F}[p]^W$ *be the minimal kernel generator of* $\mathbf{F}_+$*, so that* $z \times \mathbf{F} = (g(p), 0, \ldots, 0)$*. Let* $h(p)$ *be the GCD of all coordinates of the first row of* $\mathrm{Adj}(\mathbf{F})$*. Then,* $\det(\mathbf{F}) \equiv h(p)g(p)$ *up to an* $\mathbb{F}$*-multiple.*

Proof. Let w be the first row of $\mathrm{Adj}(\mathbf{F})$, so that $w \times \mathbf{F} = (\det(\mathbf{F}), 0, \ldots, 0)$. It follows that w lies in the left kernel of $\mathbf{F}_+$, and therefore is an $\mathbb{F}[p]$-multiple of z: $w = h(p) \cdot z$ for some $h(p)$, thus $w \times \mathbf{F} = (h(p)g(p), 0, \ldots, 0)$. Since coordinates of z are mutually coprime, it follows that $h(p)$ is equal to GCD of the coordinates of w, up to a constant multiple. □

Corollary 2. *If* $p = p_0$ *is part of a solution to the equation system represented by full-rank* $\mathbf{F} \in \mathbb{F}[p]^{W \times W}$*, then* p_0 *is a root of* $\det \mathbf{F}$*.*

We conclude that this allows us to adjust our elimination approach as follows: given an equation system represented as a full-rank square matrix over $\mathbb{F}[p]$, compute its determinant and find its roots. This shows a close relationship with (in fact, a generalization of) the algebraic FreeLunch approach [BBL+24]. Note that similarity of Corollary 2 is rather surprising, since we essentially consider a raw equation matrix and have not even defined a monomial order, while the FreeLunch technique constructs a multiplication matrix in the quotient ring (modulo a Gröbner basis).

The state-of-the-art univariate polynomial matrix determinant algorithm from [LNZ17] has complexity $\tilde{\mathcal{O}}(W^\omega \lceil \xi / W \rceil)$ (where the row-degree sum ξ can be replaced by any other determinant's degree upper bound), equal to $\tilde{\mathcal{O}}(W^{\omega-1}\xi)$ when $\xi \geq W$. This may be asymptotically better than our $\mathcal{O}((W^2 + s)\xi)$ term (in our attacks on AIM, we will have $s = \mathcal{O}(W^2)$). However, due to the hidden logarithmic factors in $\tilde{\mathcal{O}}$, we expect our method to have better concrete complexity for our (sparse) use-cases. Furthermore, one may consider unlikely that the whole row of the adjugate matrix has a common factor (especially for large fields), and in this case our algorithms essentially compute the matrix determinant, up to a constant multiple. This observation is of independent interest, and it is an interesting future work of whether our algorithm can improve previous determinant-based techniques.

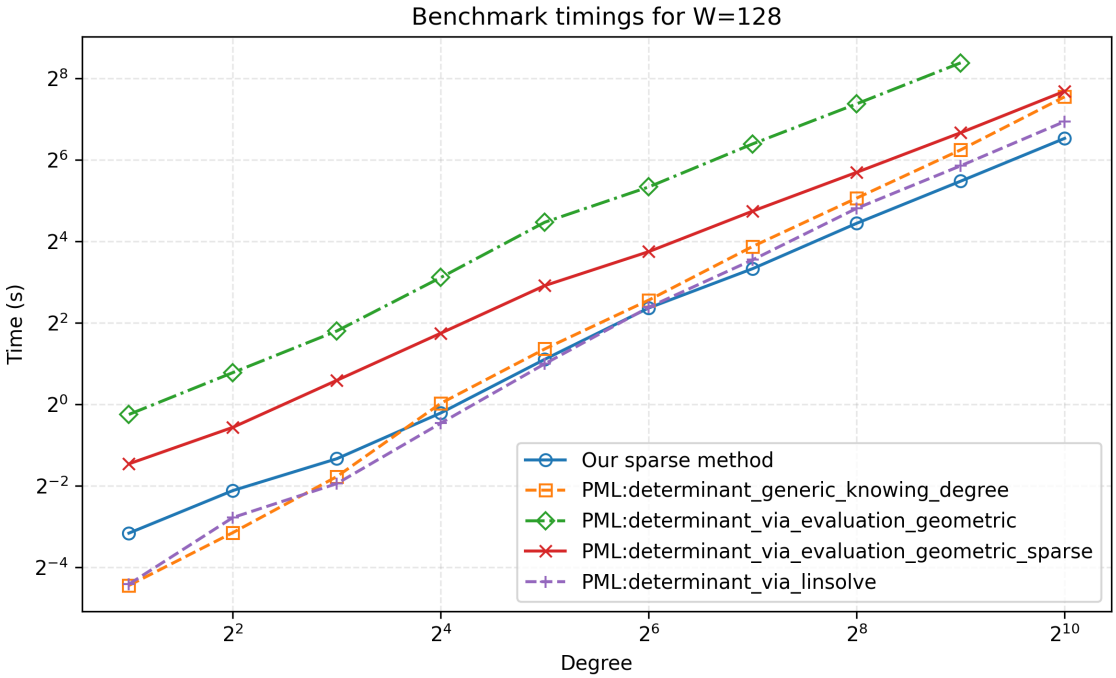


Fig. 2: Benchmark of our algorithm and the ones available in PML, on a $W \times W$ polynomial matrix and $s = 3W^2$ nonzero coefficients, $W = 128$. The graph shows time as a function of the matrix' degree.

Benchmark against PML We performed a basic comparison of our method and the methods for computing the determinant available in the PML library [HNS19,PML25] combined with the NTL library [Sho25], over a 55-bit prime field. In addition, we added an improved variant of the existing method based on evaluation over geometric series, where the first step is to evaluate the polynomial matrix at a geometric progression; we replaced it by a more efficient method based on the high sparsity of the matrix. For experiments, we generated a random $W \times W$ polynomial matrix of degree d, with 3 nonzero coefficients per entry. For different W, the ranking of different algorithms varied. For very small values of W ($W \leq 32$), the algorithms relying on linear algebra (such as the one described above) performed overall better. For larger values of W, our algorithm starts to outperform others from a certain degree. See Figure 2 for an example. More graphs are available in Section H and the supporting code.

4 Application of the new elimination technique to AIM2

In this section, we apply the new elimination framework to AIM2, improving attacks from Subsection 2.5.

4.1 Adapting the (M,L)-systems

Let $M \in \mathbb{Z}_{\geq 0}, L \in \mathbb{Z}_{>0}$, and let F be an (M, L)-system without explicit multiplications by p^j (we view it over $\mathbb{F}_{2^n}[p][t_1, \ldots, t_\ell]$). As in Definition 3, we will write $\mathbf{F} \in \mathbb{F}_{2^n}[p]^{H \times W}$ for the corresponding polynomial matrix and $\mathbf{F}_+ \in \mathbb{F}_{2^n}[p]^{H \times (W-1)}$ for its truncated version (with the $\mathbf{t}$-constant column removed). We start with simple observations about these systems.

Proposition 14. *For an (M, L)-system viewed as an $\mathbb{F}_{2^n}[p]$-module, we have*

$$\begin{aligned} W &= \ell n + 1, && \textit{(number of } \mathbf{t}\textit{-monomials)} \\ H &= (L + \ell) \cdot (M + 1), && \textit{(number of equations)} \\ s &< 2W^2. && \textit{(number of nonzero coefficients in } W \textit{ equations)} \end{aligned}$$

In particular, for $\mathbf{F}$ to have full column rank W, it is necessary that $H \geq W$:

$$(L + \ell)(M + 1) \geq \ell n + 1.$$

The sparsity bound follows from the fact that each equation (and all its squarings) in Proposition 1 has at most $2\ell n < 2W$ coefficients. It is only counted for the W selected equations (out of H) to be used in Algorithm 1.

Theorem 6. *Let F be an (M, L)-system for AIM2, for some $M, L \in \mathbb{Z}_{\geq 0}$. If the corresponding $\mathbf{F}(0)$ has full column rank W and there exist W pivot rows with row-degree sum ξ, one can find solutions to the system in time given by Theorem 5 and an extra cost of $\mathcal{O}(W^{\omega-1}H)$ field operations.*

Proof. We can sort rows of $\mathbf{F}$ by their degrees, and using fast linear algebra on $\mathbf{F}(0)$ (over $\mathbb{F}_{2^n}$, cost $\mathcal{O}(W^{\omega-1}H)$) we can find the first N pivot rows, which will minimize the row-degree sum ξ. The rest is a direct application of Theorem 5 to the square polynomial matrix. □

Our general approach is to work with a concrete cipher instance, construct an (M, L)-system, and evaluate its rank experimentally over $\mathbb{F}_{2^n}$ (by setting $p = 0$). In case of success, this also yields the minimized row-degree sum ξ, from which we can compute the complexity of the full attack without the need to run it. Although this validates the attack only for the current *instance*, we observed no changes in rank and row-degree sum in repeated experiments, and thus expect the attack to be valid in an overwhelming majority of cases. We also implemented and executed the *full attack* on a wide range of feasible parameters, providing full confidence in the complete process.

4.2 Detailed analysis

We start by showing a row-degree bound, conditional on the fact that the minimal value of L leads to full-rank system. We will assess this claim later, both experimentally and theoretically.

Proposition 15. *Let $M \in \mathbb{Z}_{\geq 0}$. Let F be an (M, L)-system for AIM2 with $L = \left\lceil \frac{\ell n+1}{M+1} - \ell \right\rceil$, the corresponding $\mathbf{F} \in \mathbb{F}_{2^n}[p]^{H \times W}$ and $\mathbf{F}_+$ defined as in Definition 3, $\operatorname{rank} \mathbf{F} = W$. Then, the row-degree sum of W pivots of $\mathbf{F}$ satisfies*

$$\xi \leq 2^{M+1} \cdot \left\lceil \frac{\ell n + 1}{M + 1} \right\rceil.$$

Proof. According to Proposition 14, the number of IVs satisfying $(L+\ell)(M+1) \geq \ell n+1$ is the smallest for a full rank to be possible. Each of the IVs (and the two base equations f_1, f_2) contributes $M+1$ equations of degrees $1, 2, \ldots, 2^M$ in p, adding up to $2^{M+1}-1$. Thus, we get the total degree sum $\xi \leq (2^{M+1}-1)(L+\ell)$ (the inequality can happen due to rounding, where some equations will be omitted as non-pivots). □

Corollary 3. *Let $\ell \in \{2, 3\}$, $M = \left\lceil \frac{\ell n+1}{\ell+L} \right\rceil - 1$. Let F be an (M, L)-system for AIM2 with the corresponding $\mathbf{F} \in \mathbb{F}_{2^n}[p]^{H \times W}$ and $\mathbf{F}_+$ defined as in Definition 3. If $\mathbf{F}(0)$ has full rank, the secret key for AIM2 can be recovered in time (in field operations)*

$$\sim 21 \cdot 2^{\frac{\ell n}{\ell+L}} (\ell + L) n^3$$

Proof. Observe that $W = \ell n + 1$ and $H = (\ell + 1)(M + 1) = \mathcal{O}(n)$, so that the cost of finding pivots $\mathcal{O}(W^\omega)$ is negligible. Based on Theorem 6, we compute the cost given in Theorem 5. From Proposition 15, we get the bound (note $\ell + L \geq 3$)

$$\xi \;\leq\; 2^{\frac{\ell n+1}{\ell+L}} \cdot (\ell + L) \;\leq\; 1.26 \cdot 2^{\frac{\ell n}{\ell+L}} \cdot (\ell + L), \qquad \log_2 \xi \leq n + \mathcal{O}(\log(\ell + L)).$$

We ignore the last term as negligible, since $L < n$ in our applications. By Proposition 14, we have $s \leq 2W^2$ nonzero coefficients. We obtain time complexity (Theorem 5)

$$\begin{aligned}
&\sim \xi \cdot (4W^2 + 4s) + \mathbf{M}(\xi) \cdot (2n + 2W + 22 \log_2 \xi) \\
&\leq 1.26 \cdot 2^{\frac{\ell n}{\ell+L}} (\ell + L) \left(12(\ell n + 1)^2 + 0.5 \cdot n^2 (2n + 2\ell n + 2 + 22 \cdot n)\right) \\
&\leq 21 \cdot 2^{\frac{\ell n}{\ell+L}} (\ell + L) n^3 \quad \text{(for } n \geq 128\text{)}.
\end{aligned}$$

□

Proof of the system's full rank We continue with a formal proof that the system has full rank in the case of a single IV. While we have verified this by practical experiments, a theoretical proof shows that such analysis is possible and the framework is not purely empirical. This section provides high-level ideas of the proof; full proof with additional technical details is given in Section F. We will need the following lemma.

Lemma 2. *Let $2 \leq e < n$ be coprime with n, and let $a \in \mathbb{F}_{2^n} \setminus \{0\}$. Then, the map $\phi_a : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n} : t \mapsto t^{2^e} + a \cdot t$ has $\mathbb{F}_2$-rank $n-1$ and kernel $\{0, a^{(2^e-1)^{-1}}\}$. Therefore, there exists a bijective $\mathbb{F}_{2^n}$-linear change of monomials*

$$\left(t, t^2, t^4, t^8, \ldots, t^{2^{n-1}}\right) \quad \Leftrightarrow \quad \left(\tilde{t}, \tilde{t}^2, \tilde{t}^4, \ldots, \tilde{t}^{2^{n-2}}, b\right)$$

for $\tilde{t} = \phi_a(t)$, $b = \operatorname{Tr}(\alpha \cdot t)$, and some $\alpha, \in \mathbb{F}_{2^n} \setminus \{0\}$.

Proof. Observe that $t^{2^e} + at = t(t^{2^e-1} - a) = 0$ has roots 0 and all $(2^e - 1)^{\text{th}}$ roots of a. If $\operatorname{GCD}(2^e - 1, 2^n - 1) = 2^{\operatorname{GCD}(e,n)} - 1 = 1$, then $2^e - 1$ does not divide the multiplicative group order $2^n - 1$ and there exists only one required root of a in $\mathbb{F}_{2^n}$, namely a^d where $d \equiv (2^e - 1)^{-1} \pmod{2^n - 1}$.

We now arbitrarily choose $\alpha \in \mathbb{F}_{2^n}$ with $\operatorname{Tr}(\alpha a^d) = 1$ and define an $\mathbb{F}_2$-linear subspace $V \subseteq \mathbb{F}_{2^n}$ as $V = \{u \in \mathbb{F}_{2^n} \mid \operatorname{Tr}(\alpha u) = 0\}$ such that $\mathbb{F}_{2^n} = V \oplus \{0, a^d\}$. Clearly, all $\tilde{t}^{2^i}$ are $\mathbb{F}_2$-linear functions of t by

definition, similarly for b. Since all $\tilde{t}^{2^i}$ are outputs of a rank-$(n-1)$ linear map, they satisfy a relation of the form $\mathrm{Tr}(\delta\tilde{t}) = \sum_{i=0}^{n-1} \delta^{2^i}\tilde{t}^{2^i} = 0$, for a nonzero $\delta \in \mathbb{F}_{2^n}$. Therefore, $\tilde{t}^{2^{n-1}}$ can be expressed as an $\mathbb{F}_{2^n}$-combination of all the other $\tilde{t}^{2^i}$. Since the kernel of the map ϕ_a restricted to V is zero (due to the rank consideration), there exists its linear inverse A, which is computable as an $\mathbb{F}_{2^n}$-linear combination from all $\tilde{t}^{2^i}$. Then, the full input is recovered from the decomposition $\mathbb{F}_{2^n} = V \oplus \{0, a^d\}$ where the second component is simply derived from b. □

Theorem 7. *Let $n \geq 16$, $M+1 = \left\lceil \frac{\ell n+1}{\ell+1} \right\rceil$. Then, an $(M,1)$-system over $\mathbb{F}_{2^n}[p]$ with $c \neq 0$ has full rank with probability at least $1 - 2^{\frac{-n+3}{\ell+1}-1} = 1 - \mathcal{O}\left(2^{-\frac{n}{\ell+1}}\right)$.*

Proof. When considering the system modulo p, we get three groups of polynomials (for $\ell = 2$): $t_1^{2^{e_1+i}}, t_2^{2^{e_2+i}} + t_2^{2^i}, \mathrm{Lin}(\mathrm{t}_1,\mathrm{t}_2)^{2^{e_*+\mathrm{i}}} + \mathrm{cLin}(\mathrm{t}_1,\mathrm{t}_2)^{2^{\mathrm{i}}}$, $0 \leq i \leq M$. Using Lemma 2, we define the variable $\tilde{t}_2 = \phi_1(t_2)$ and the second group transforms into single monomials. The key idea is then to eliminate the monomials given by the first two groups ($t_1^{2^i}$, $\tilde{t}_2^{2^j}$ for $2(M+1)$ values of i,j). We then focus on the structure of the third group without the eliminated monomials. It consists of a submatrix described by $\phi_c(\mathrm{Lin}(\mathrm{t}_1,\mathrm{t}_2))$, which is a kind of circulant Moore matrix, built from random variables induced by the random Lin layer. We then show that determinant of the remaining submatrix is a nonzero degree-2^{M+1} polynomial of those variables, and by the Schwartz-Zippel lemma obtain the desired upper bound on the determinant (note $M \approx 2n/3$ and so $2^{M+1} \ll 2^n$). □

n	ℓ	L (IVs)	M	W (monom.)	ξ (deg. sum)	Time	Memory	Experiment
128	2	127	1	257	$2^{8.59}$	$2^{27.30}$	$2^{19.71}$	(29 seconds)
128	2	15	15	257	$2^{19.25}$	$2^{38.44}$	$2^{28.26}$	(1 hour)
128	2	2	64	257	$2^{66.32}$	$2^{88.64}$	$2^{75.33}$	-
128	2	1	85	257	$2^{87.32}$	$2^{110.66}$	$2^{96.33}$	-
128	3	1	96	385	$2^{98.32}$	$2^{122.26}$	$2^{107.91}$	-
128	4	1	102	513	$2^{105.00}$	$2^{129.32}$	$2^{115.00}$	-
192	2	191	1	385	$2^{9.17}$	$2^{29.02}$	$2^{20.88}$	(72 seconds)
192	2	31	11	385	$2^{16.78}$	$2^{36.83}$	$2^{26.39}$	(35 minutes)
192	2	2	96	385	$2^{98.32}$	$2^{122.32}$	$2^{107.91}$	-
192	2	1	128	385	$2^{130.00}$	$2^{155.04}$	$2^{139.59}$	-
192	3	1	144	577	$2^{146.32}$	$2^{171.97}$	$2^{156.49}$	-
192	4	1	153	769	$2^{156.17}$	$2^{182.19}$	$2^{166.76}$	-
192	5	1	160	961	$2^{162.81}$	$2^{189.10}$	$2^{173.72}$	-
192	6	1	164	1153	$2^{167.58}$	$2^{194.09}$	$2^{178.76}$	-
256	3	382	1	769	$2^{10.17}$	$2^{31.97}$	$2^{22.87}$	(10 minutes)
256	3	94	7	769	$2^{14.54}$	$2^{36.38}$	$2^{25.34}$	(38 minutes)
256	3	2	153	769	$2^{156.17}$	$2^{182.23}$	$2^{166.76}$	-
256	3	1	192	769	$2^{194.32}$	$2^{221.19}$	$2^{204.91}$	-
256	4	1	204	1025	$2^{207.32}$	$2^{234.55}$	$2^{218.32}$	-
256	5	1	213	1281	$2^{216.17}$	$2^{243.67}$	$2^{227.49}$	-
256	6	1	219	1537	$2^{222.46}$	$2^{250.17}$	$2^{234.05}$	-

Table 4: Attacks on full AIM2. The time complexity is in field operations. For large (non-standard) ℓ, we have chosen the new exponents at random (conditioned on the S-box invertibility).

Experiments Usually, the main difficulty when dealing with attacks by linearization lies in estimating the linear independence correctly. In our case, using Proposition 8, we can easily verify the rank of random instances in practice by dealing with $(\ell n+1) \times (\ell n+1)$ matrices, obtained by substituting $p = 0$ into the original large-degree yet sparse polynomial matrices. Table 4 shows the results obtained for various

parameters, including AIM2 instances with an increased value of ℓ to evaluate possible design repairs. For all of the instances presented, we experimentally verified the full rank of $\mathbf{F}$ (using the substitution $p = 0$) and calculated the theoretical complexity based on the actual observed value of ξ, typically just slightly lower than the estimate given by Proposition 15. This part of the experiment was not heavily optimized, but it always finished in a couple of minutes, and its complexity is polynomial in n, ℓ, and L. Furthermore, for those cases with practically feasible running time, we ran the full key recovery attack (with randomly generated key and IVs), which requires dealing with the full degree-ξ matrix. The last column presents the time taken for the key recovery part, which had a sufficient optimization effort put into.[7]

Multiplications by t-monomials We also considered additional multiplications by monomials in the t_i variables, increasing the allowed algebraic degree in them from one to two or more, in line with traditional XL. Our initial estimations based on the number of equations and monomials suggested potential improvements. However, experimentally, we observed many linear dependencies (over $\mathbb{F}_{2^n}[p]$) so that the full rank of the matrix $\mathbf{F}$ is reached roughly at the same degree of p as in the attacks described above. Since, at the same time, the dimension of the matrix W increases, the complexities only grow. More details are available in Section G. Our conclusion is that this technique is not useful, at least for the model considered of AIM2. However, it will be necessary to improve attacks on 3-round RAIN in Section 5, where the initial system already has quadratic algebraic degree in the auxiliary variables $\mathbf{t}$.

5 Application to Rainier/RAIN

Description of Rainier/RAIN Rainier [DKR+22] is a signature scheme similar to AIMer. It is based as well on computing the public key from the private key through a specifically developed block cipher, RAIN. It has six models, according to two parameters: number of rounds ($r = 3, 4$), and security level ($n = 128, 192, 256$), which also defines the finite field in which operations take place ($\mathbb{F}_{2^n}$). The underlying block cipher has either three or four rounds, consisting of an addition of the key and a constant, the S-box (multiplicative inversion), and a random $\mathbb{F}_2$-linear layer. The final round makes a final addition of the key instead of the linear layer. The scheme for 3 rounds is represented in Figure 3, together with the intermediate variables used in our attacks.

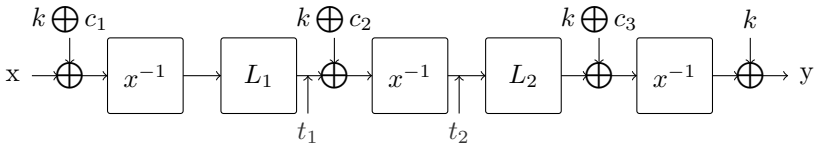


Fig. 3: RAIN, with $r = 3$ rounds

x is the public input, L_1, L_2 different linear layers, x^{-1} the finite field inversion, c_1, c_2, c_3 are public constants, and k is the key. t_1, t_2 are intermediate variables that we introduce for our attack.

Algebraic modeling From the figure, the following algebraic models for 2 and 3 rounds are derived (we assume that none of the intermediate values equals zero):

2 rounds

$$g_1: \ (x + k + c_1) \cdot L_1^{-1}(t_1) = 1,$$
$$g_2: \ (t_1 + k + c_2) \cdot (k + y) = 1.$$

3 rounds

$$f_1: \ (x + k + c_1) \cdot L_1^{-1}(t_1) = 1,$$
$$f_2: \ (t_1 + k + c_2) \cdot t_2 = 1,$$
$$f_3: \ (L_2(t_2) + k + c_3) \cdot (k + y) = 1.$$

We briefly describe attacks on 2 and 3 rounds of RAIN, using the same methodology as for AIM, except customized monomial multiplications. We compute the rank and the degree sum experimentally and then derive complexity estimates directly using our existing theory.

[7] When comparing timings with simple linearization attacks presented in Subsection 2.5, one has to take into account that those were executed using an unoptimized SageMath implementation as a proof-of-concept.

(RAIN-2) We start with the two equations g_1, g_2 from the model described before and consider the system of equations

$$g_1^{2^m}, \quad (g_1 \cdot L_1^{-1}(t))^{2^m}, \quad g_2^{2^m}, \quad (g_2 \cdot t_1)^{2^m},$$

for each $m \in [0, M]$, for a parameter M.

Experimentally, we have always observed the full rank at $M = n/2 - 1$ yielding $\xi = 2^{n/2+2}$, the same univariate polynomial degree as in [YZY26], obtained with a different method. This results in attack complexities (in field operations) $2^{87.04}$, $2^{120.75}$, and $2^{153.97}$ for $n = 128, 192$, and 256, respectively. Note that our complexities are higher than in [YZY26] partly due to the different estimation of the univariate root finding costs.

(RAIN-3) For the 3-round RAIN, the initial model contains a monomial $t_1 t_2$ complicating the process. The idea is to allow the set of monomials of the shapes $t_1^{2^i} t_2^{2^j}$ for $(i, j) \in [0, M] \times [0, n]$ and $(i, j) \in [0, n] \times [0, M]$ for a chosen parameter $M < n$, and to derive as many equations as possible within this monomial restriction. For each $m \in [1, M]$ and for each $j \in [0, M]$ we consider the following equations:[8]

$$\begin{aligned}
&f_1^{2^m}, \ (f_1 \cdot L_1^{-1}(t_1))^{2^m}, \ (f_1 \cdot L_1^{-1}(t_1))^{2^m} t_2^{2^j}, \ f_1 t_2^{2^j}, \\
&f_2^{2^m}, \ (f_2 \cdot t_1)^{2^m}, (f_2 \cdot t_1 \cdot t_2)^{2^m}, \\
&(f_3 \cdot L_2(t_2))^{2^m}, \ (f_3 \cdot L_2(t_2))^{2^m} t_1^{2^j}, \ f_3 t_1^{2^j}.
\end{aligned}$$

Experimentally, the full rank for $n = 128$ is achieved with $M = 109$, $W = 16\,318$ monomials, row-degree sum $\xi = 2^{115.78}$, and the time complexity of $2^{146.07}$ field operations. This computation took 1.5 hours and 42 GB of RAM. For $n = 192$ the computation was more heavy (28 hours and 322 GB of RAM), yielding an attack with $M = 170$, $\xi = 2^{175.61}$, time complexity of $2^{208.24}$ field operations and memory complexity of $2^{196.37}$ bytes.

6 Conclusions

In this work, we developed a new algebraic elimination framework and applied it to AIMer v2.1 / AIM2, where we also discovered practical attacks in scenarios where the same secret key is reused under different IVs or specifically related secret keys are used.

The key-reuse weakness arises from the fact that the IV does not affect the whitening constants; the related-key weakness, however, stems from the two-branch setting. Therefore, both these special scenarios can be avoided by making constants IV-dependent and using at least 3 branches. An alternative way is to explicitly require users to change the secret key each time they change their public key.

However, our elimination framework also yields strong attacks in the standard single-key single-IV setting, suggesting an inherent weakness of the design. In order to counter the attacks, the number of branches needs to be increased to at least $\ell = 4, 6, 7$ respectively for the three versions (in addition to making the constants IV-dependent). This estimate is based on the number of field operations and does not include any additional security margin, and is likely to impose a significant performance overhead.

Our techniques also lead to improved algebraic attacks on 3-round (full) RAIN; however, they do not beat the exhaustive search yet, leaving the design unbroken. The key difference with AIM is the serial application of S-boxes instead of the parallel layer. This introduces multiplication of internal variables, while the essentially flat structure of AIM is linear (since p is treated as a parameter of the system). We conclude that the sequential structure from RAIN is more secure, which is also evidenced by many cryptanalysis efforts.

Acknowledgements

Work funded by Luxembourg's FNR projects CryptoFin (C22/IS/17415825), PQseal (C24/IS/18978392), NCER22/IS/16570468/NCER-FT, and FinnovationHub funded by the Ministry of Finance, Government

[8] Some groups of equations fitting the monomial shapes were manually excluded since they do not increase the rank.

of Luxembourg. We thank the designers of AIM2 and the anonymous reviewers for fruitful discussions and suggestions.

References

AAB+22. Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jerome Lacan, Jean-Marc Robert, and Pascal Veron. HQC. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions. 1

BBL+24. Augustin Bariant, Aurélien Boeuf, Axel Lemoine, Irati Manterola Ayala, Morten Øygarden, Léo Perrin, and Håvard Raddum. The algebraic FreeLunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IV*, volume 14923 of *LNCS*, pages 139–173. Springer, Cham, August 2024. doi:10.1007/978-3-031-68385-5_5. 2, 4, 16

BBLP22. Augustin Bariant, Clémence Bouvier, Gaëtan Leurent, and Léo Perrin. Algebraic attacks against some arithmetization-oriented primitives. *IACR Trans. Symm. Cryptol.*, 2022(3):73–101, 2022. doi:10.46586/tosc.v2022.i3.73-101. 4, 15, 25

BCG+24. Daniel J. Bernstein, Jolijn Cottaar, Emanuele Di Giandomenico, Kathrin Hövelmanns, Andreas Hülsing, Mikhail Kudinov, Tanja Lange, Mairon Mahzoun, Matthias Meijers, Alex Pellegrini, Alberto Ravagnani, Silvia Ritsch, Sven Schäge, Tianxin Tang, Monika Trimoska, Marc Vorstermans, and Fiona Johanna Weber. Report on evaluation of KpqC round-2 candidates. Cryptology ePrint Archive, Report 2024/2077, 2024. URL: https://eprint.iacr.org/2024/2077. 3, 4

BGFU26. Alex Biryukov, Pablo García Fernández, and Aleksei Udovenko. Magic pot: Cryptanalysis of full AIM2 in the standard and related-/reused-key settings using new elimination framework - supporting code, 2026. doi:10.5281/zenodo.18727305. 4

BNSED22. Jérémy Berthomieu, Vincent Neiger, and Mohab Safey El Din. Faster change of order algorithm for gröbner bases under shape and stability assumptions. In *ISSAC 2022*, page 409–418, New York, NY, USA, 2022. ACM. doi:10.1145/3476446.3535484. 4, 9

CCD+24. Jung Hee Cheon, Hyeongmin Choe, Julien Devevey, Tim Güneysu, Dongyeon Hong, Markus Krausz, Georg Land, Marc Möller, Damien Stehlé, and MinJune Yi. HAETAE: Shorter lattice-based fiat-shamir signatures. *IACR TCHES*, 2024(3):25–75, 2024. doi:10.46586/tches.v2024.i3.25-75. 1

CCHY24. Jung Hee Cheon, Hyeongmin Choe, Dongyeon Hong, and MinJune Yi. SMAUG: Pushing lattice-based key encapsulation mechanisms to the limits. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *SAC 2023*, volume 14201 of *LNCS*, pages 127–146. Springer, Cham, August 2024. doi:10.1007/978-3-031-53368-6_7. 1

CKPS00. Nicolas Courtois, Alexander Klimov, Jacques Patarin, and Adi Shamir. Efficient algorithms for solving overdefined systems of multivariate polynomial equations. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 392–407. Springer, Berlin, Heidelberg, May 2000. doi:10.1007/3-540-45539-6_27. 4, 7

CP02. Nicolas Courtois and Josef Pieprzyk. Cryptanalysis of block ciphers with overdefined systems of equations. In Yuliang Zheng, editor, *ASIACRYPT 2002*, volume 2501 of *LNCS*, pages 267–287. Springer, Berlin, Heidelberg, December 2002. doi:10.1007/3-540-36178-2_17. 2, 4, 7

DKR+22. Christoph Dobraunig, Daniel Kales, Christian Rechberger, Markus Schofnegger, and Greg Zaverucha. Shorter signatures based on tailor-made minimalist symmetric-key crypto. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 843–857. ACM Press, November 2022. doi:10.1145/3548606.3559353. 1, 20

GG13. Joachim von zur Gathen and Jürgen Gerhard. *Modern computer algebra*. Cambridge University Press, Cambridge, third edition. edition, 2013. 11, 24, 25

GJV03. Pascal Giorgi, Claude-Pierre Jeannerod, and Gilles Villard. On the complexity of polynomial matrix computations. In *ISSAC 2003*, page 135–142, New York, NY, USA, 2003. ACM. doi:10.1145/860854.860889. 10, 11

GM10. Shuhong Gao and Todd Mateer. Additive Fast Fourier Transforms over finite fields. *IEEE Transactions on Information Theory*, 56(12):6265–6272, 2010. doi:10.1109/TIT.2010.2079016. 4, 24

HBD+22. Andreas Hülsing, Daniel J. Bernstein, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, Jean-Philippe Aumasson, Bas Westerbaan, and Ward Beullens. SPHINCS+. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. 1

HNS19. S. G. Hyun, V. Neiger, and É. Schost. Implementations of efficient univariate polynomial matrix algorithms and application to bivariate resultants. In *Proceedings ISSAC 2019*, pages 235–242. ACM, 2019. https://github.com/vneiger/pml. doi:10.1145/3326229.3326272. 17

JNV20. Claude-Pierre Jeannerod, Vincent Neiger, and Gilles Villard. Fast computation of approximant bases in canonical form. *Journal of Symbolic Computation*, 98:192–224, 2020. Special issue on Symbolic and Algebraic Computation: ISSAC 2017. doi:10.1016/j.jsc.2019.07.011. 10, 11

KHS+23. Seongkwang Kim, Jincheol Ha, Mincheol Son, ByeongHak Lee, Dukjae Moon, Joohee Lee, Sangyub Lee, Jihoon Kwon, Jihoon Cho, Hyojin Yoon, and Jooyoung Lee. AIM: Symmetric primitive for shorter signatures with stronger security. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 401–415. ACM Press, November 2023. doi:10.1145/3576915.3616579. 1, 4

KHS+24. Seongkwang Kim, Jincheol Ha, Mincheol Son, ByeongHak Lee, Dukjae Moon, Joohee Lee, Sangyub Lee, Jihoon Kwon, Jihoon Cho, Hyojin Yoon, and Jooyoung Lee. The AIMer signature scheme. Version 2.1. *Samsung, S. D. S.*, 2024. URL: https://aimer-signature.org/docs/AIMer-specification-v2.1.pdf. 1, 4

KHSL23. Seongkwang Kim, Jincheol Ha, Mincheol Son, and Byeonghak Lee. Efficacy and mitigation of the cryptanalysis on AIM. Cryptology ePrint Archive, Report 2023/1474, 2023. URL: https://eprint.iacr.org/2023/1474. 1

KP22. Jonghyun Kim and Jong Hwan Park. NTRU+: Compact construction of NTRU using simple encoding method. Cryptology ePrint Archive, Report 2022/1664, 2022. URL: https://eprint.iacr.org/2022/1664. 1

LCH14. Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to Reed-Solomon erasure codes. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pages 316–325, 2014. doi:10.1109/FOCS.2014.41. 24

LCK+18. Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius Additive Fast Fourier Transform. In *ISSAC 2018*, page 263–270, New York, NY, USA, 2018. ACM. doi:10.1145/3208976.3208998. 4, 13, 24

LDK+22. Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. 1

LMM24. Fukang Liu, Mohammad Mahzoun, and Willi Meier. Modelling ciphers with overdefined systems of quadratic equations: Application to Friday, Vision, RAIN and Biscuit. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part VII*, volume 15490 of *LNCS*, pages 424–456. Springer, Singapore, Singapore, December 9–13, 2024. doi:10.1007/978-981-96-0941-3_14. 3, 4

LMØM23. Fukang Liu, Mohammad Mahzoun, Morten Øygarden, and Willi Meier. Algebraic attacks on RAIN and AIM using equivalent representations. *IACR Trans. Symm. Cryptol.*, 2023(4):166–186, 2023. doi:10.46586/tosc.v2023.i4.166-186. 1, 3, 4

LN96. Rudolf Lidl and Harald Niederreiter. *Finite Fields*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2 edition, 1996. 29

LNZ17. George Labahn, Vincent Neiger, and Wei Zhou. Fast, deterministic computation of the Hermite normal form and determinant of a polynomial matrix. *J. Complex.*, 42:44–71, 2017. doi:10.1016/J.JCO.2017.03.003. 16

NSS+17. Matus Nemec, Marek Sys, Petr Svenda, Dusan Klinec, and Vashek Matyas. The return of coppersmith's attack: Practical factorization of widely used RSA moduli. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1631–1648. ACM Press, October / November 2017. doi:10.1145/3133956.3133969. 2

PML25. The PML team. *PML: Polynomial Matrix Library*, 2025. Version 0.5, https://github.com/vneiger/pml. 17

SAB+22. Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022. 1

Sch77. Arnold Schönhage. Schnelle multiplikation von polynomen über körpern der charakteristik 2. *Acta Informatica*, 7(4):395–398, Dec 1977. doi:10.1007/BF00289470. 24

SD25. The Sage Developers. *SageMath, the Sage Mathematics Software System (Version 10.7)*, 2025. URL: https://www.sagemath.org. 4

Sho25. Victor Shoup. *NTL: A Library for doing Number Theory*, 2025. Version 11.6.0, https://libntl.org. 17

SS71. Arnold Schönhage and Volker Strassen. Schnelle multiplikation grosser zahlen. *Computing*, 7(3):281–292, 1971. 24

vdH25. Joris van der Hoeven. Optimizing the half-gcd algorithm. *Applicable Algebra in Engineering, Communication and Computing*, May 2025. doi:10.1007/s00200-025-00690-w. 4, 11, 14, 25

Wie86. Douglas H. Wiedemann. Solving sparse linear equations over finite fields. *IEEE Trans. Inf. Theory*, 32(1):54–62, 1986. doi:10.1109/TIT.1986.1057137. 8

Yap99. Chee-Keng Yap. Lecture ii: The gcd. Lecture notes (PDF), September 1999. Dated September 9, 1999. URL: https://people.eecs.berkeley.edu/~fateman/282/readings/yap-2.pdf. 24

YZY26. Hong-Sen Yang, Qun-Xiong Zheng, and Jing Yang. Algebraic cryptanalysis of ao primitives based on polynomial decomposition. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025*, pages 383–410, Singapore, 2026. Springer Nature Singapore. doi:10.1007/978-981-95-5018-0_13. 2, 3, 4, 21

ZCD+20. Greg Zaverucha, Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, Jonathan Katz, Xiao Wang, Vladmir Kolesnikov, and Daniel Kales. Picnic. Technical report, National Institute of Standards and Technology, 2020. available at https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions. 2

ZLS12. Wei Zhou, George Labahn, and Arne Storjohann. Computing minimal nullspace bases. In *ISSAC 12*, page 366–373, New York, NY, USA, 2012. ACM. doi:10.1145/2442829.2442881. 10, 11

ZWY+23. Kaiyi Zhang, Qingju Wang, Yu Yu, Chun Guo, and Hongrui Cui. Algebraic attacks on round-reduced rain and full AIM-III. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part III*, volume 14440 of *LNCS*, pages 285–310. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8727-6_10. 1, 3, 4

A Complexity of polynomial operations

A.1 Multiplication of polynomials

Theorem 8 (Schönhage-Strassen multiplication, [SS71,Sch77],[GG13, Ex.8.30]). *Two polynomials of degree at most d over a commutative ring R where 3 is a unit can be multiplied in at most*

$$15.12 \cdot d \log_2 d \log_2 \log_2 d + \mathcal{O}(d \log d)$$

operations in R.

Theorem 9 ([GM10, Sec.III]). *Two polynomials of degree at most d over a field $\mathbb{F}$ of characteristic 2 can be multiplied in at most*

$$0.5 \cdot d(\log_2 d)^2 + \mathcal{O}(d \log d)$$

operations in $\mathbb{F}$.

Note that the dominating part of the algorithm is the change of basis, which only involves additions in the field. The number of multiplications is in $\mathcal{O}(n \log n)$ and the hidden constant is, in fact, below 6.

We will consider degrees around $2^{2/3n}$ for n between 128 and 256, where the additive FFT still outperforms the Schönhage-Strassen method. However, Gao-Mateer also proposed a second method which has complexity $\sim d \log_2 d \log_2 \log_2 d$ but which only works for degrees of the form $d = 2^{2^k}$ for some integer k. One can speculate that this method can be generalized and applied for all degrees.

For practical implementation, we adapted the approach and the implementation of Frobenius Additive FFT [LCK+18].

Proposition 16. *Two degree-d polynomials over $\mathbb{F}_{2^n}$ can be multiplied with complexity $\mathbf{M}(d) = \mathcal{O}(d \log_2 d)$ when $d = 2^{2^k}$ for an integer k, and $\mathbf{M}(d) \sim 0.5 \cdot d \log_2^2 d$ in general, using Gao-Mateer's Additive FFT [GM10] or its optimized variant called Frobenius Additive FFT (FAFFT) [LCK+18].*

The complexity is dominated by a change of polynomial basis. Although [LCK+18] claims complexity $\mathcal{O}(d \log_2 d \log_2 \log_2 d)$ in general, it only refers to [GM10] and [LCH14], which state that this complexity is reached when the degree is a double power of 2. We also note that this complexity is reached by [Sch77], but it is expected to have a high constant factor. In our analysis, we will use the general cost from [GM10].

A.2 GCD and HalfGCD

HalfGCD of two non-constant degree-d polynomials $f(x), g(x)$ returns the middle Euclidean remainder $r(x), \deg r(x) < d/2$ and its polynomial Bézout coefficients $u(x), v(x)$ of degree at most $\lceil d/2 \rceil$: $u \cdot f + v \cdot g = r$. In particular, $r \neq 0$ if f and g are coprime. Polynomial GCD can be computed using calls to HalfGCD with degrees d, $d/2$, $d/4$, …, implying total double cost compared to HalfGCD. For details we refer to [GG13,Yap99].

Proposition 17. *The complexity cost of HalfGCD is* $\sim 2.75\mathbf{M}(n)\log_2(n)$*, according to [vdH25] (we consider the general FFT case). The complexity cost of GCD is* $\sim 5.5\mathbf{M}(n)\log_2(n)$.

A.3 Univariate root finding

The roots of a univariate polynomial $f(x) \in \mathbb{F}[x]$ can be found using the standard method (see e.g. [BBLP22]): compute $\mathrm{GCD}(x^q - x, f(x))$, where q is the size of the field $\mathbb{F}$, which is equal to the product of $(x - r_i)$ for all roots r_i of f belonging to $\mathbb{F}$. This is usually done by raising x to the power q while working modulo $f(x)$ and then subtracting x, followed by GCD of the result with $f(x)$.

Proposition 18. *Assuming* $f(x)$ *has a small number of roots in* $\mathbb{F}$*,* $\deg f = d$*, the cost of finding them is given by*

$$\sim \mathbf{M}(d)(6\log_2|\mathbb{F}| + 11\log_2 d).$$

If $\mathbb{F} = \mathbb{F}_{2^n}$*, the cost reduces to*

$$\sim \mathbf{M}(d)(2n + 11\log_2 d).$$

Proof. The computation of $x^q - x$ modulo $f(x)$ using binary exponentiation requires at most $2\log_2|\mathbb{F}|$ polynomial multiplications ($\mathbf{M}(d)$) and reductions (each of which costs $\sim 2\mathbf{M}(d)$ [GG13, Thm.9.6] with precomputation of $f^{-1} \mod x^d$ costing $\mathcal{O}(\mathbf{M}(d))$). For the binary field, multiplications (squarings) are linear time, and the field size is a power of two, thus we only need n modular reductions.

The GCD computation has cost $11\mathbf{M}(d)\log_2 d$ similarly to the analysis in Proposition 11.

Since the number of roots is assumed to be small, factoring the resulting polynomial can be done using auxiliary techniques in negligible time. □

B Deterministic version of the polynomial kernel vector computation

We describe a deterministic version of Algorithm 2. It is based on the following idea. Recall that we are looking for a kernel vector $v \in \mathbb{F}_q[p]$ of degree at most ξ given its polynomial multiple $z = v \cdot h(p) \mod p^o$, $o = 2\xi + 2$. We assume that z is normalized by a multiplication with the inverse of the first coordinate (modulo p^o). The first Padé approximation (computed using $\mathrm{HalfGCD}(z_2, p^o)$, follows up by a division by the full GCD) finds the minimal representation of z_2 as a fraction r/u, $\deg r < o/2$, equivalent to v_2/v_1. The minimality of the representation means that any common factors of v_2 and v_1 are removed. However, it may be possible that some of these factors do not divide other coordinates v_i. The idea is therefore to recompute and multiply these missing factors by successive Padé approximations with the remaining coordinates. More precisely, every z_j is represented as r_j/u_j minimally, which yields the factor u_j to be multiplied to the full vector. Lemma 1 provides a basis for this analysis.

Algorithm 3 Kernel vector of a polynomial matrix of nullity 1, robust version

Inputs: matrix $G \in \mathbb{F}_q[p]^{(N+1)\times N}$ with s nonzero coefficients in total, $\mathrm{rank}\, G(0) = N$
Output: vector $z \in \mathbb{F}_q[p]^{N+1}_{\leq\xi}$ such that $z \times G = 0 \in \mathbb{F}_q[p]^N$ and $\xi = \sum_{i=1}^{N+1} \deg G_i$

1: $o \leftarrow 2\xi + 2$
2: $z \leftarrow \textsc{ApproximantVector}(G, o)$ ▷ Algorithm 1
3: $z \leftarrow z/(z_1^{-1} \pmod{p^o})$ ▷ Can be skipped, Algorithm 1 ensures $z_1 = 1$
4: **for all** $i \in \{2, \ldots, N+1\}$ **do**
5: $\quad (u, v) \leftarrow \mathrm{HalfGCD}(z_j, p^o)$ ▷ $u \cdot z_j + v \cdot p^o = r$, $\deg u \leq o/2, \deg r < o/2$
6: $\quad u' \leftarrow u/\mathrm{GCD}(u, v)$
7: $\quad z \leftarrow u' \cdot z$
8: **return** z

C Comparison of the two attacks

Figure 4 illustrates the time complexity evolution of the two attacks in terms of the number L of IVs: by direct linearization (Subsection 2.5) and by polynomial matrix-based elimination (Subsection 4.2), on the example of AIM2-I.

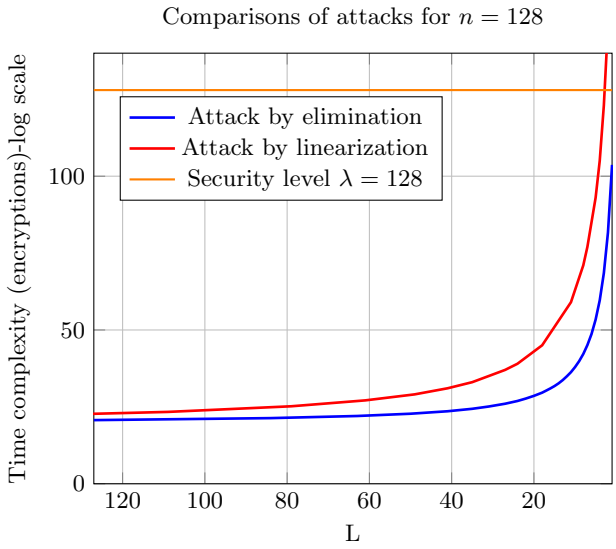


Fig. 4: Comparison of the attack by linearization (Subsection 2.5) with the attack by elimination (Section 4). Both attacks are considered under the conditions $D = 0$, $n = 128$.

D Rank analysis for the linearization attack

In this section, we provide partial rank analysis for an (M, L)-system for AIM2 ($D = 0$ - no multiplications by t_i), relevant for the basic linearization attack (Subsection 2.5).

Theorem 10. *Let $F_1 : t_1^{2^{e_1}} = pt_1$, let sq, j be nonnegative integers such that $2^{sq} + j \leq 2^m$, for $m < n-1$. Then, when we consider the system of equations created by all combinations $F_1^{2^{sq}} \cdot p^j$, the corresponding linear independence is maximal.*

Proof. For equation eq, we will call the right-hand side $eq - r$, and the left-hand side $eq - l$. Since the general formula for one of these equations is

$$eq : t_1^{2^{sq+e_1}} p^j = p^{2^{sq}+j} t_1^{2^s q},$$

they establish equalities among monomials. The maximum linear independence rank is obtained unless there exists a cycle, that is, a series of equations $eq_1, \ldots, eq_t$ such that

$$eq_1 - r/l = eq_1 - l/r = eq_2 - l/r = eq_2 - r/l = \cdots = eq_t - r/l = eq_t - l/r.$$

To check for cycles, first we have to look for collisions of monomials in different equations. Given $eq_1(sq, j)$, $eq_2(sq', j')$, we have three types of collisions:

1. $p^{2^{sq}+j} t_1^{2^{sq}} = p^{2^{sq'}+j} t_1^{2^{sq}} \Rightarrow eq_1 = eq_2 \ (sq = sq', j = j')$
2. $t_1^{2^{sq+e_1}} p^j = t_1^{2^{sq'+e_1}} p^{j'} \Rightarrow eq_1 = eq_2$
3. $t_1^{2^{sq+e_1}} p^j = p^{2^{sq'}+j} t_1^{2^{sq}} \Rightarrow sq' = sq + e_1, j = 2^{sq'} + j' = 2^{sq+e_1} + j'$

We just have to study the last case. When this happens, we have one possible cycle (due to the increase in the value of the squaring exponent):

$$\cdots = p^{2^{sq^{(3)}}+j^{(3)}} t_1^{2^{sq^{(3)}}} = t_1^{2^{sq'+e_1}} p^{j'} = p^{2^{sq'}+j} t_1^{2^{sq}} = t_1^{2^{sq+e_1}} p^j = p^{2^{sq}+j} t_1^{2^{sq}} = t_1^{2^{sq''+e_1}} p^{j''} = \ldots \quad (7)$$

Since $\gcd(e_1, n) = 1$, we would need all n exponents to close a cycle, so m should be at least $n - 1$. □

We have calculated the rank for $n = 128, 192, 256$ and various values of m, and the results are as expected.

Note that the equations on t_2 and $\mathrm{Lin}(\mathrm{t}_1, \mathrm{t}_2)$ are linearly equivalent to the shape of the equation on t_1. Therefore, the same statement applies to them, separately. Furthermore, the equations on t_1 and t_2 are clearly linearly independent together, since variables cannot cancel each other (and there is no constant equation in the linear span of any of the two groups, since otherwise the system is inconsistent).

However, we could not yet prove full rank of all three equation groups together, due to the uncertainty arising from random linear layer connecting the two variables.

E Proof of the basis change proposition

Below we provide the proof of Proposition 5.

Proposition 5. *Let $y \in \mathbb{F}_{2^n}$ be a variable. Then, there exist n variables $\hat{y}_0, \ldots, \hat{y}_{n-1} \in \mathbb{F}_2$ and an $\mathbb{F}_{2^n}$-linear bijection between vectors*

$$(y, y^2, y^4, y^8, \ldots, y^{2^{n-1}}) \quad \textit{and} \quad (\hat{y}_0, \ldots, \hat{y}_{n-1}).$$

Proof. Let w be an element of $\mathbb{F}_{2^n}$ such that $\{w^{2^i}\}_{i=0}^{n-1}$ are linearly independent over $\mathbb{F}_2$. We can set

$$\hat{y}_j = \mathrm{Tr}(w \cdot y^{2^j}) = \sum_{i=0}^{n-1} w^{2^i} y^{2^{i+j}} \in \mathbb{F}_2.$$

Then, we just have to extend the function linearly. The $\mathbb{F}_2$-linear independence propagates through this function. For any $\lambda_0, \ldots, \lambda_{n-1} \in \mathbb{F}_2$, we have:

$$\sum_{i=0}^{n-1} \lambda_i \hat{y}_i = \sum_{i=0}^{n-1} \lambda_i \sum_{j=0}^{n-1} w^{2^j} y^{2^{i+j}} = \sum_{k=0}^{n-1} \sum_{\substack{i,j:i+j=k \\ (\mathrm{mod}\ n)}} w^{2^j} \lambda_i y^{2^k} = 0 \Leftrightarrow \sum_{i,j:j+i=k} \lambda_i w^{2^j} = 0$$

for every k. In particular, when $k = n-1$, we have $\sum_{j=0}^{n-1} \lambda_{n-1-j} w^{2^j} = 0$. By the independence of these elements, all the λ_k must be zero, which implies that the traces preserve linear independence and, therefore, the function is injective. Lastly, since the cardinality of the domain coincides with that of its codomain, by the rank-nullity theorem, the function must be a bijection. □

F Proof of the full-rank polynomial matrix (single IV)

Below we provide the proof of Theorem 7. We will also use an extension of Lemma 2.

Lemma 3. *Let $2 \leq e < n$ be coprime with n, let $c \in \mathbb{F}_{2^n} \setminus \{0\}$ and define the map $\phi_c : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n} : t \mapsto t^{2^e} + c \cdot t$. There exists an $\mathbb{F}_2$-linear bijection B and nonzero constants $\alpha, \beta, s_\alpha, s_\beta \in \mathbb{F}_{2^n}$ such that*

$$\phi_c(B(x)) = x + s_\beta \cdot \mathrm{Tr}(\beta x),$$
$$B(\phi_c(x)) = x + s_\alpha \cdot \mathrm{Tr}(\alpha x).$$

Proof. Recall from Lemma 2 that ϕ_c is bijective on the subspace V given by $\mathrm{Tr}(\alpha x) = 0$, $\alpha \neq 0$, and has kernel $\{0, s_\alpha\}$, $s_\alpha = c^{(2^e-1)^{-1}}$, $\mathrm{Tr}(\alpha s_\alpha) = 1$.

Let $W = \phi_c(V)$ be the image of V, described by $\mathrm{Tr}(\beta x) = 0$ for some nonzero $\beta \in \mathbb{F}_{2^n}$. Let nonzero $s_\beta \in \mathbb{F}_{2^n}$ be arbitrary such that $\mathrm{Tr}(\beta s_\beta) = 1$.

Since ϕ_c is injective on V, we can deduce the $\mathbb{F}_2$-linear map $B : W \to V$ such that $\phi_c(B(x)) = x$ for all $x \in W$. We extend it to $B : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$ by setting $B(s_\beta) = s_\alpha$. Observe that B is a bijection, since

$W + \{0, s_\beta\} = V + \{0, s_\alpha\} = \mathbb{F}_{2^n}$, $B(W) = V$ by the initial definition, and $B(s_\beta) = s_\alpha$ by the extension. Pictorially,

$$\begin{array}{ccccc} W + \{0, s_\beta\} \simeq \mathbb{F}_{2^n} & \xrightarrow{B} & V + \{0, s_\alpha\} \simeq \mathbb{F}_{2^n} & \xrightarrow{\phi_c} & W + \{0, s_\beta\} \simeq \mathbb{F}_{2^n}, \\ y_0 + b \cdot s_\beta & \mapsto & x_0 + b \cdot s_\alpha & \mapsto & y_0. \end{array}$$

Let $y = y_0 + b \cdot s_\beta$ for $y_0 \in W, b \in \mathbb{F}_2$. Notice that $\mathrm{Tr}(\beta y) = b$. Then,

$$\begin{aligned} \phi_c(B(y)) &= \phi_c(B(y_0 + b \cdot s_\beta)) \\ &= \phi_c(B(y_0)) + \phi_c(B(b \cdot s_\beta)) = y_0 + b \cdot \phi_c(B(s_\beta)) = y_0 = y + b \cdot s_\beta \\ &= y + s_\beta \cdot \mathrm{Tr}(\beta y), \end{aligned} \tag{8}$$

where we used the fact that $b \in \mathbb{F}_2$ propagates through $\mathbb{F}_2$-linear maps. Substituting $x = B(y)$, we obtain

$$\phi_c(x) = B^{-1}(x) + s_\beta \cdot \mathrm{Tr}(\beta B^{-1}(x)).$$

Applying B to both sides, we get

$$B(\phi_c(x)) = x + B(s_\beta) \cdot \mathrm{Tr}(\beta B^{-1}(x)) = x + s_\alpha \cdot \mathrm{Tr}(\beta B^{-1}(x)).$$

From $B^{-1}(V) = W$ and $W = \{u \in \mathbb{F}_{2^n} \mid \mathrm{Tr}(\beta u) = 0\}$ we deduce $\mathrm{Tr}(\beta B^{-1}(x)) = \mathrm{Tr}(\alpha x)$. □

Theorem 7. *Let $n \geq 16$, $M + 1 = \left\lceil \frac{\ell n + 1}{\ell + 1} \right\rceil$. Then, an $(M, 1)$-system over $\mathbb{F}_{2^n}[p]$ with $c \neq 0$ has full rank with probability at least $1 - 2^{\frac{-n+3}{\ell+1}-1} = 1 - \mathcal{O}\left(2^{-\frac{n}{\ell+1}}\right)$.*

Proof. The equations from Proposition 1 (with $\gamma_1 = 0, \gamma_2 = 1$ by Proposition 3) can be written as

$$t_1^{2^{e_1+i}} \equiv 0 \pmod{p}, \quad (f_1^{2^i}) \tag{9}$$

$$t_2^{2^{e_2+i}} + t_2^{2^i} \equiv 0 \pmod{p}, \quad (f_2^{2^i}) \tag{10}$$

$$L^{2^{e_*+i}} + c^{2^i} \cdot L^{2^i} \equiv 0 \pmod{p}, \quad (f_3^{2^i}) \tag{11}$$

where $L = \mathrm{Lin}(\mathrm{t}_1, \mathrm{t}_2)$ and $0 \leq i \leq M$. Note that we do not consider them as equations per se, but only as polynomials, because generally $p \neq 0$.

(High-level plan) The determinant of the equation matrix is a multivariate polynomial in the coefficients $\lambda_{i,j}$ of the affine map Lin (viewed as a linearized polynomial). Its degree is at most $\sum_{i=0}^{M} 2^i = 2^{M+1} - 1$, since, in the Leibniz formula, each monomial takes only one variable from each i-th row of the third group, and these variables have degree at most 2^i. We now show that this polynomial is nonzero by exemplifying Lin achieving full rank (i.e., choosing concrete Lin arbitrarily). The theorem will then follow from the Schwartz-Zippel lemma applied to the determinant: the determinant is equal to zero with probability at most $2^{M+1}/2^n$, over the assignments of the variables $\lambda_{i,j}$, i.e., over the choice of Lin. Since we only consider Lin consisting of bijective components per branch, the probability may grow by at most a factor of 4^ℓ (a random square $n \times n$ matrix over $\mathbb{F}_2$ has full rank with probability around $0.29 \geq 1/4$). Using $M \leq \lceil (\ell n + 1)/(\ell + 1) \rceil$, we obtain the probability upper bound

$$4^\ell 2^{M+1-n} \leq 2^{2\ell + \frac{\ell n+1}{\ell+1} + 1 - n} = 2^{\frac{1-n}{\ell+1} + 2\ell + 1} = \mathcal{O}\left(2^{-\frac{n}{\ell+1}}\right).$$

(Substitutions) We will apply Lemma 2 and Lemma 3 twice, first to $\phi_1(x) = x^{2^{e_2}} + x$, and then to $\phi_c(x) = x^{2^{e_*}} + c \cdot x$ (note that we require $c \neq 0$). We will use subscripts 1 and c to distinguish the corresponding linear maps B and elements $\alpha, \beta, s_\alpha, s_\beta$.

First, we replace the variables $t_2, t_2^2, \ldots, t_2^{2^{n-1}}$ with new variables $\tilde{t}_2, \tilde{t}_2^2, \ldots, \tilde{t}_2^{2^{n-2}}$, $\mathrm{Tr}(\alpha_1 t_2)$ via the inverse of the substitution $\tilde{t}_2 = t_2^{2^{e_2}} + t_2$, bijective, and $\mathbb{F}_{2^n}$-linear by Lemma 2/Lemma 3:

$$t_2 = B_1(\tilde{t}_2) + s_{\alpha_1} \cdot \mathrm{Tr}(\alpha_1 t_2).$$

The second polynomial group $f_2^{2^i}$ simply becomes $\tilde{t}_2^{2^i}$, $0 \le i \le M$ (note that $\tilde{t}_2^{2^{n-1}}$ does not appear since $i \le M < n-1$). We also substitute $\tilde{t}_1 = t_1^{e_1}$ for the purpose of symmetry, obtaining polynomials $\tilde{t}_1^{2^i}$ in the first group $f_1^{2^i}$. Rewriting the third base equation as $f_3 = \phi_c(L) = \phi_c(L_1(t_1)) + \phi_c(L_2(t_2)) + \phi_c(\gamma)$, we get

$$f_3 = \phi_c(L_1(\tilde{t}_1)) + \phi_c\Big(L_2\Big(B_1(\tilde{t}_2) + s_{\alpha_1} \cdot \mathrm{Tr}(\alpha_1 t_2)\Big)\Big) + \phi_c(\gamma),$$

where $L(t_1, t_2) = L_1(t_1) + L_2(t_2) + \gamma$ with linear maps L_1, L_2 and constant γ, which we will now choose to obtain a full rank instance of the system.

(Choosing Lin*)* We now set

$$\begin{aligned} L_1(t_1) &= B_c(\tilde{t}_1^{2^{M-h-2}}), \\ L_2(t_2) &= B_c(B_1^{-1}(t_2)^{2^{M-2}}) = B_c(\tilde{t}_2^{2^{M-2}}) + s'_{\alpha_1} \cdot \mathrm{Tr}(\alpha_1 t_2)), \end{aligned}$$

where $h = n - 2 - M \approx n/3 \approx M/2$ and $s'_{\alpha_1} = L_2(s_{\alpha_1})$ is a nonzero constant. Using $\phi_c(B(x)) = x + s_{\alpha_c} \cdot \mathrm{Tr}(\alpha_c x)$, the third base equation becomes

$$f_3 \;=\; \tilde{t}_1^{2^{M-h-2}} \;+\; \tilde{t}_2^{2^{M-2}} \;+\; s_{\alpha_c} \;\cdot\; \mathrm{Tr}(\alpha_c(\tilde{t}_1^{2^{M-h-2}} \;+\; \tilde{t}_2^{2^{M-2}})) \;+\; \phi_c(s'_{\alpha_1}) \;\cdot\; \mathrm{Tr}(\alpha_1 t_2) \;+\; \phi_c(\gamma).$$

(Monomial basis) In order to avoid the expansion of $v = \mathrm{Tr}(\alpha_c(\tilde{t}_1^{2^{M-h-2}} + \tilde{t}_2^{2^{M-2}}))$, we substitute $\tilde{t}_1^{2^{n-1}}$ by v. This is a bijective $\mathbb{F}_{2^n}$-linear change, since the trace expression necessarily contains $\tilde{t}_1^{2^{n-1}}$ and all the other monomials are available (note that $\tilde{t}_2^{2^{n-1}}$ is expressible from $\tilde{t}_2, \ldots, \tilde{t}_2^{2^{n-2}}$ by Lemma 2): the substitution simply adds other monomials to $\tilde{t}_1^{2^{n-1}}$ with the right scaling. The final set of $2n+1$ monomials is

$$\begin{aligned} &\tilde{t}_1, \tilde{t}_1^2, \ldots, \tilde{t}_1^{2^{n-2}}, && (n-1 \text{ monomials}) \\ &\tilde{t}_2, \tilde{t}_2^2, \ldots, \tilde{t}_2^{2^{n-2}}, && (n-1 \text{ monomials}) \\ &\mathrm{Tr}(\alpha_c(\tilde{t}_1^{2^{M-2}} + \tilde{t}_2^{2^{M-h-2}})),\ \mathrm{Tr}(\alpha_1 t_2),\ 1. && (3 \text{ monomials}) \end{aligned}$$

(Elimination process) We can now run an elimination process showing that, for every monomial in our basis, there exists an $\mathbb{F}_{2^n}$-linear combination of equations yielding exactly this monomial. The $(2M+2)$ monomials $\tilde{t}_1, \ldots, \tilde{t}_1^{2^M}$ and $\tilde{t}_2, \ldots, \tilde{t}_2^{2^M}$ are already given by the first two groups; they can be eliminated from the other equations. In the following, we consider the equations $f_3^{2^i}$, $0 \le i \le M$. The process is illustrated in Figure 5.

1. Case ($0 \le i \le 2, 3$ monomials): The monomials $\tilde{t}_1^{2^{M-h-2+i}} + \tilde{t}_2^{2^{M-2+i}}$ can be eliminated using the first two groups of equations (containing monomials with exponents up to the power 2^M). For the last three monomials in the basis, we obtain the matrix

$$\begin{bmatrix} s_{\alpha_c} & \phi_c(s'_{\alpha_1}) & \phi_c(\gamma) \\ s_{\alpha_c}^2 & \phi_c(s'_{\alpha_1})^2 & \phi_c(\gamma)^2 \\ s_{\alpha_c}^4 & \phi_c(s'_{\alpha_1})^4 & \phi_c(\gamma)^4 \end{bmatrix}.$$

   This is a Moore matrix, and by Moore's lemma [LN96, Lem.3.51] it has full rank if the first row elements are $\mathbb{F}_2$-independent. In the definition of L_2 we can insert a constant multiplication between B_1^{-1} and B_c. This preserves the shape of the equations and changes s'_{α_1}. Similarly, we can assign γ to obtain arbitrary $\phi_c(\gamma)$ up to one bit (the trace $\mathrm{Tr}(\alpha_c \gamma)$). Therefore, it is easy to make the first three elements $\mathbb{F}_2$-linearly independent. We conclude that this 3×3 submatrix is of full rank and the corresponding monomials can be extracted. From now on, we can eliminate them from the other equations.

2. Case ($3 \le i \le h+2, h$ monomials): We can still eliminate all $\tilde{t}_1$-related monomials that occur: the exponent of $\tilde{t}_1^{2^{M-h-2+i}}$ ranges from $M-h-2 \approx n/3 \ge 0$ to M, as well as the three previously

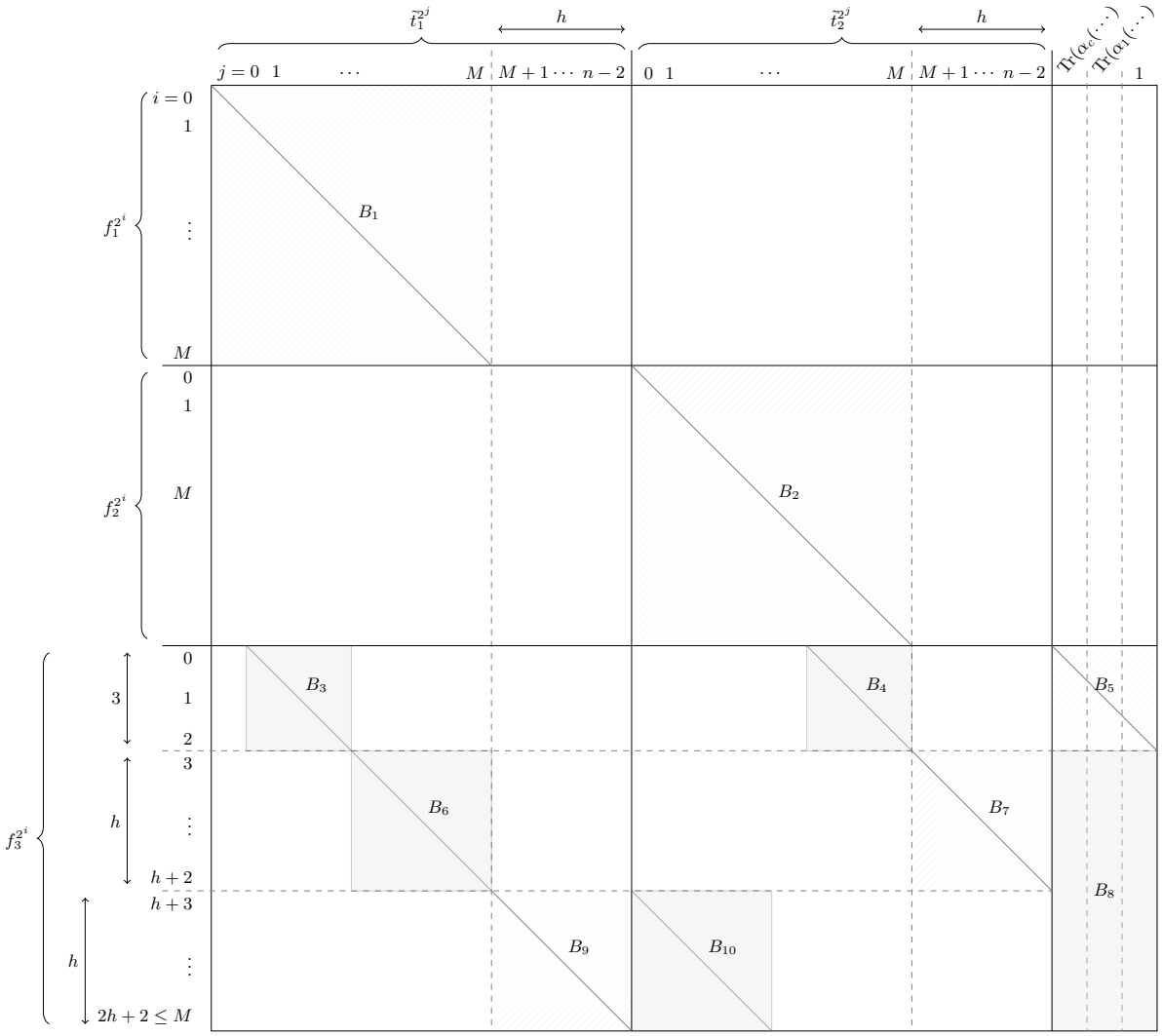


Fig. 5: Elimination process in the proof of Theorem 7.

recovered monomials (we recall that by "recovering" we mean computing a linear combination of equations equal to the single monomial; we are not recovering the value of the monomial, which is not even defined). We are thus recovering $\tilde{t}_2^{2^{M-2+i}}$: from $\tilde{t}_2^{2^{M+1}}$ up to $\tilde{t}_2^{2^{M+h}} = \tilde{t}_2^{2^{n-2}}$.

3. Case ($h+3 \le i \le 2h+2, h$ monomials): We can now eliminate all $\tilde{t}_2$-related monomials: $\tilde{t}_2, \ldots, \tilde{t}_2^{2^n-2}$ are recovered, and $\tilde{t}_2^{2^{n-1}}$ is an $\mathbb{F}_{2^n}$-linear combination of those. We are thus recovering $\tilde{t}_1^{2^{M-h-2+i}}$: from $\tilde{t}_2^{2^{M+1}}$ up to $\tilde{t}_1^{2^{M+h}} = \tilde{t}_1^{2^{n-2}}$. Finally, the monomial $\tilde{t}_1^{2^{n-1}}$ is obtained by combining the just recovered monomials and the monomial $\mathrm{Tr}(\alpha_c(\tilde{t}_1^{2^{M-h-2}} + \tilde{t}_2^{2^{M-2}}))$.

   We also need to show that $2h+2 \le M$, i.e., that we have enough polynomials $f_3^{2^i}$ for the elimination process. Since $h = n-2-M$, $2h+2 \le M$ if and only if $2n-2 \le 3M$. We have

$$3M \;=\; 3\left\lceil \frac{2n+1}{2+1} \right\rceil - 3 \;\ge\; 3\frac{2n+1}{3} - 3 \;=\; 2n-2.$$

We conclude that each of the $2n+1$ monomials can be obtained by an $\mathbb{F}_{2^n}$-linear combination, and therefore, the equation matrix has full rank for the given choice of Lin.

Generalization to larger ℓ The construction naturally generalizes to larger ℓ. We sketch an example for $\ell = 3$. We would need a new substitution $\tilde{t}_3 = t_3^{2^{e_3}} + \gamma_3 t_3$, which still fits our lemma. We will recover monomials in a similar reverse order: first the special ones (trace-based and the constant), then $\tilde{t}_3^{2^i}$, then $\tilde{t}_2^{2^i}$, and, finally, $\tilde{t}_1^{2^i}$. The increased $M \approx n - n/(\ell+1)$ naturally allows to accommodate the "inactive" powers (i.e., those recoverable by the other groups of polynomials). The last base equation becomes

$$\begin{aligned} f_{\ell+1} = \tilde{t}_1^{2^{M-2h-2}} + \tilde{t}_2^{2^{M-h-2}} + \tilde{t}_3^{2^{M-2}} \\ + s_{\alpha_c} \cdot \mathrm{Tr}(\alpha_c(\tilde{t}_1^{2^{M-2h-2}} + \tilde{t}_2^{2^{M-h-2}} + \tilde{t}_3^{2^{M-2}})) \\ + \phi_c(s'_{\alpha_1}) \cdot \mathrm{Tr}(\alpha_1 t_2) + \phi_c(s'_{\alpha_{\gamma_3}}) \cdot \mathrm{Tr}(\alpha_{\gamma_3} t_3) + \phi_c(\gamma). \end{aligned}$$

In the first elimination stage, as long as $\ell \ll n$, we can easily guarantee the $\mathbb{F}_2$-linear independence in the first row, and thus, the full rank of the submatrix for special monomials. The second elimination stage is repeated for $\tilde{t}_3$ and for $\tilde{t}_2$. The final elimination stage is similar.

□

G Extending the system with multiplications by monomials in t_i

We now extend our definition of (M, L)-system to allow multiplications by monomials in the $\mathbf{t}$ variables of algebraic degree up to a parameter $D \in \mathbb{Z}_{\ge 0}$, in spirit of the XL approach. Formally, we augment Definition 1 to define a (D, M, L)-system as

$$\begin{aligned} S_{D,M,L} = \left\{ f_i^{2^m} \cdot p^j \cdot t_1^{u_1} \cdot \ldots \cdot t_\ell^{u_\ell} \right\}_{i,m,j,u_1,\ldots,u_\ell} &\subseteq \mathbb{F}[t_1, \ldots, t_\ell, p], \\ i \in \{1, \ldots, \ell+L\}, \quad m \in \{0, \ldots, M\}, \quad j \in \{0, \ldots, 2^M - 2^m\}, \\ u_1, \ldots, u_\ell \in \mathbb{Z}_{\ge 0},\ \mathrm{HW}(u_1) + \ldots + \mathrm{HW}(u_\ell) \le D. \end{aligned}$$

We now report the results of applying our framework to this case represented by setting $D = 1, 2$, whereas previous attacks from Subsection 2.5 and Subsection 4.2 correspond to $D = 0$.

We have performed several experiments to compute the actual rank of the systems obtained with $D = 1$ for AIM-I. Their outcomes are shown in Table 5. Full rank of the system with single IV ($L = 1$) is obtained only for $M = 84$, with $\xi = 2^{89.13}$, and a complexity of $2^{122.75}$ field operations. Since M is essentially the same as for $D = 0$, we conclude that $D = 1$ does not provide any benefit. This rank computation (of a $32\,897 \times 32\,897$ matrix over $\mathbb{F}_{2^{128}}$) took just a few hours to complete (with a non-optimized algorithm). Thus, even if high D is not relevant for AIM, it can be interesting as an attack idea for other protocols (as we will see in the case of RAIN).

L (IVs)	M	Rank	W
64	1	25460	32897
32	3	26145	32897
2	42	32040	32897
1	84	32897	32897

Table 5: Rank computation experiments for AIM-I and $D = 1$.

In order to confirm the validity of such an attack with a large number of multiplications, we have successfully run the full key recovery procedure for a reduced version with $n = 64, D = 1, M = 1, L = 62$, which completed in 4.5 hours.

For $D = 2$, we expect the rank deficiency to be even more severe. However, even in this case, we have been able to compute the rank for the reduced setting of $n = 32$, where we need M at least equal to 20 to obtain maximum rank with $L = 1$ IV. Again, this value matches $M \approx 2n/3$ required for our $D = 0$ attack, suggesting that $D = 2$ could not bring about any improvement (if this ratio M/n persists for larger n).

We conclude that increasing D does not seem to lead to stronger attacks. This suggests that the best strategy for our framework is to push as much complexity as possible toward the univariate variable p. However, in the next section, we will show that, in the 3-round RAIN, setting $D = 1$ with a custom set of monomials leads to a better attack. This is explained by the fact that the initial equations for RAIN already contain quadratic monomials in the $\mathbf{t}$ variables.

H Benchmarks against PML

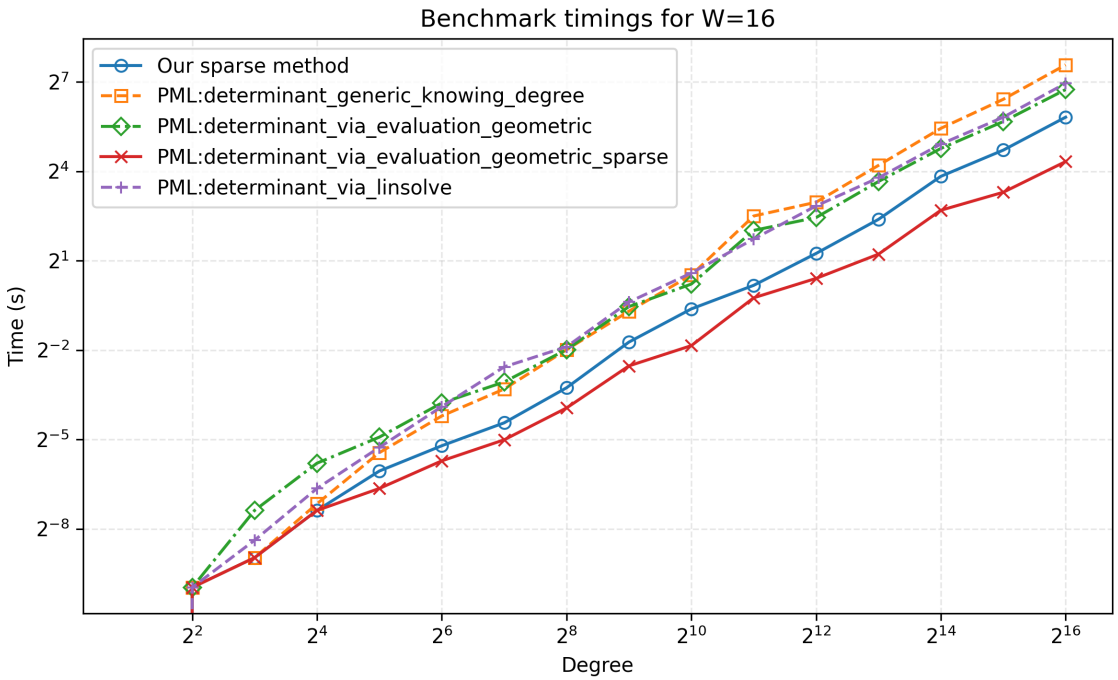


Fig. 6: Benchmark of our algorithm and the ones available in PML, on a $W \times W$ polynomial matrix and $s = 3W^2$ nonzero coefficients, $W = 16$. The graph shows time as a function of the matrix' degree.

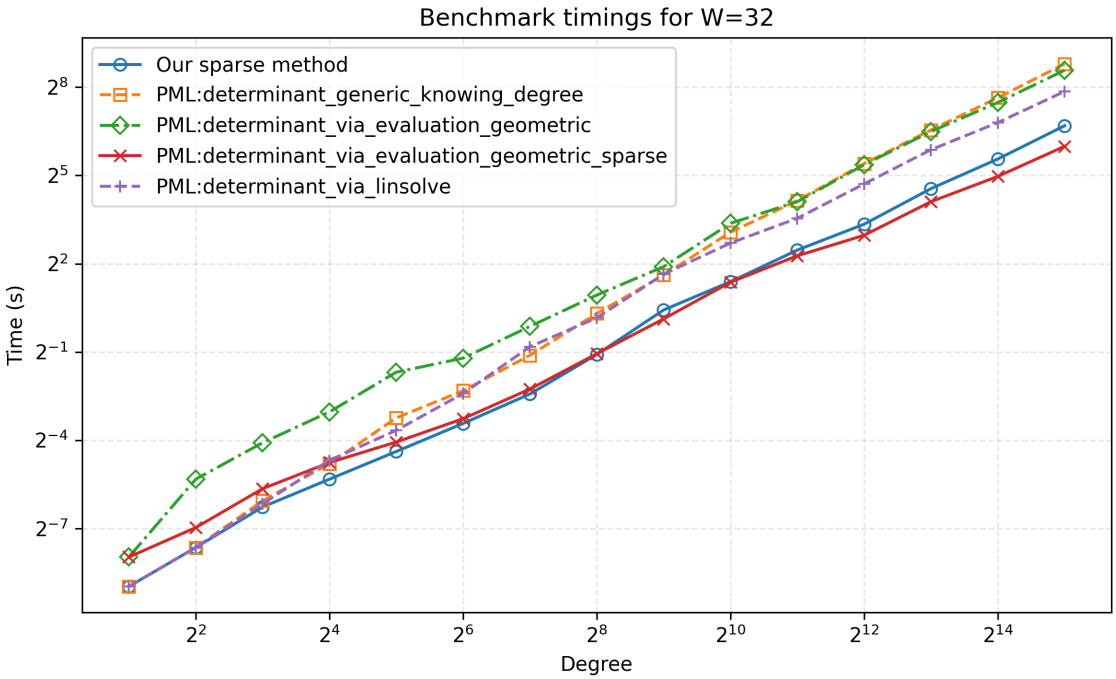


Fig. 7: Benchmark of our algorithm and the ones available in PML, on a $W \times W$ polynomial matrix and $s = 3W^2$ nonzero coefficients, $W = 32$. The graph shows time as a function of the matrix' degree.

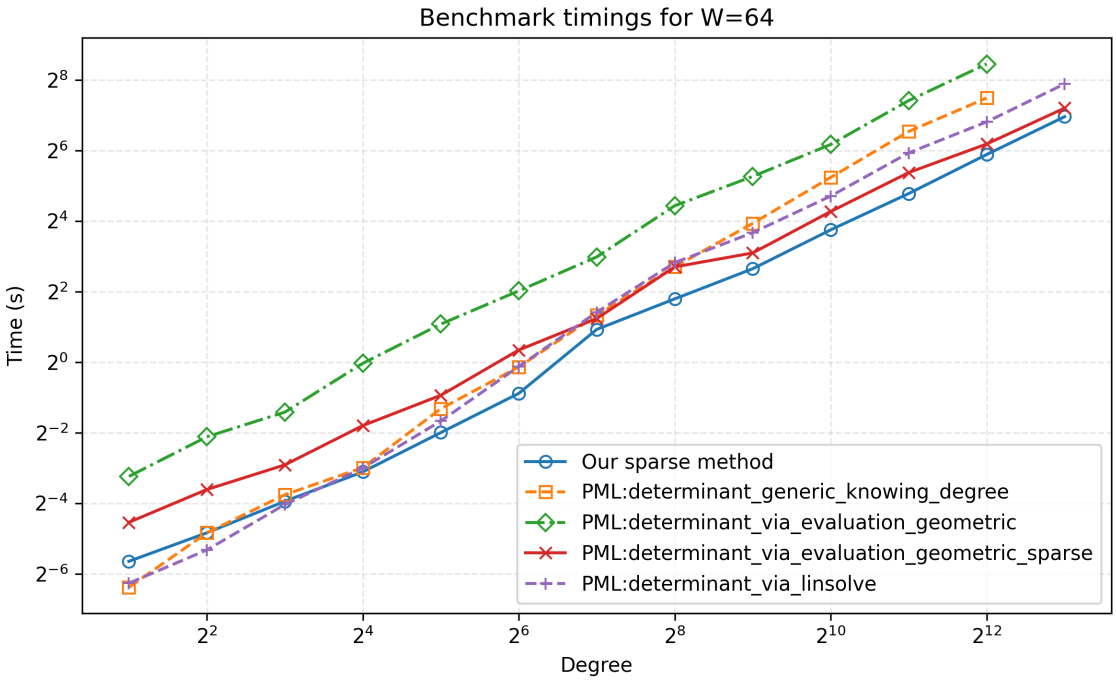


Fig. 8: Benchmark of our algorithm and the ones available in PML, on a $W \times W$ polynomial matrix and $s = 3W^2$ nonzero coefficients, $W = 64$. The graph shows time as a function of the matrix' degree.

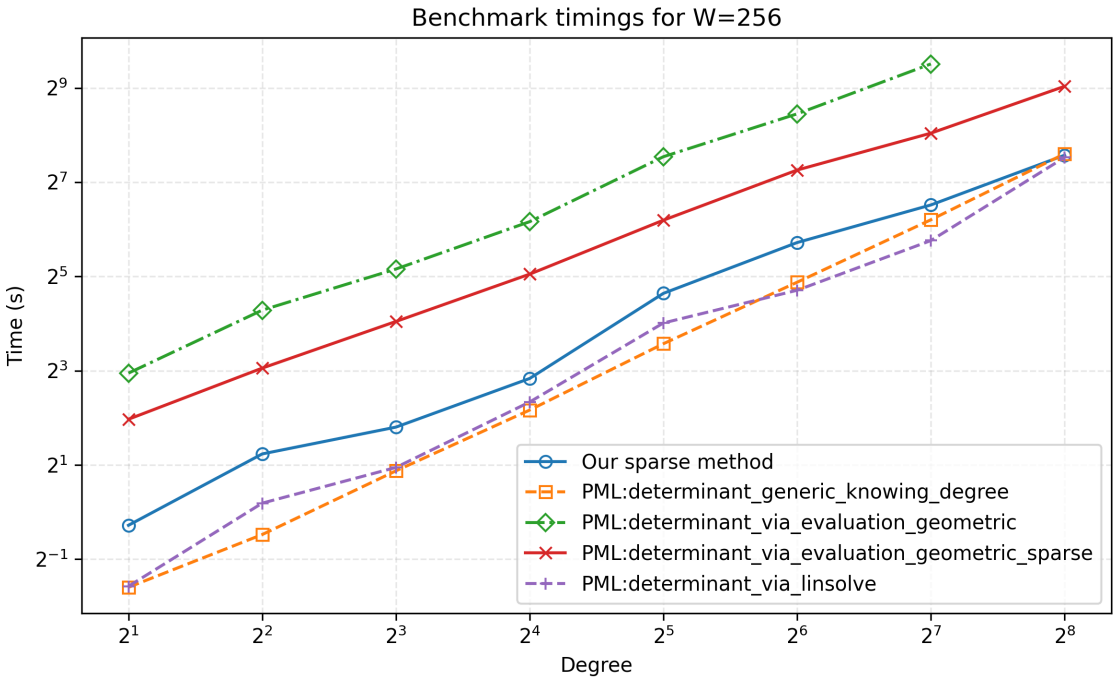


Fig. 9: Benchmark of our algorithm and the ones available in PML, on a $W \times W$ polynomial matrix and $s = 3W^2$ nonzero coefficients, $W = 256$. The graph shows time as a function of the matrix' degree.